



UNIVERSIDAD AUTÓNOMA DEL
ESTADO DE QUINTANA ROO

Introducción a IoT

Manual de Prácticas: Sensores y Actuadores

ACP-137- Introducción a IoT

AFDE-PPR-23- Principios de IoT

Manual de prácticas para la Unidad II-
Elementos de un sistema IoT

Autores:

M.M. Jesús Orifiel Álvarez Ruiz
Dr. Víctor Manuel Sánchez Huerta
Dr. Freddy Ignacio Chan Puc
Dr. Homero Toral Cruz

División de Ciencias, Ingeniería y Tecnología

Junio de 2025

ÍNDICE

Práctica Guiada 1. Sensores digitales: movimiento y proximidad	6
Práctica Guiada 2. La pantalla LCD.....	25
Práctica Guiada 3. La pantalla OLED.....	37
Práctica Guiada 4. Almacenamiento en medios extraíbles.	60
Práctica Guiada 5. Control de motores de CD y comunicación Serial.	74
Práctica Guiada 6. Sensor de temperatura y humedad en una red LAN.	85

ÍNDICE DE FIGURAS

Figura 1.1-Funcionamiento del sensor HC-SR04	8
Figura 1.2-Funcionamiento del sensor HC-SR04	9
Figura 1.3-Diagrama de pulsos del sensor HC-SR04 (Morgan, 2014).....	9
Figura 1.4-Diagrama de pulsos del sensor HC-SR04	10
Figura 1.5-Imágenes infrarrojas obtenidas con cámara termográfica.	11
Figura 1.6-El lente Fresnel (FLOTANTE, 2025).....	12
Figura 1.7-Modo de detección del sensor PIR	12
Figura 1.8-Rango de detección del sensor PIR HC-SR501 (FLOTANTE, 2025).	13
Figura 1.9-Potenciómetros de ajuste del sensor PIR HC-SR501 (FLOTANTE, 2025).....	13
Figura 1.10-Elementos del sensor PIR HC-SR501 (FLOTANTE, 2025).....	14
Figura 1.11-Sensor Inductivo LJ12A3-4-Z/BX	15
Figura 1.12-Sensor Inductivo LJ12A3-4-Z/BX (Llamas, 2025).....	16
Figura 1.13-Conexión del sensor Inductivo LJ12A3-4-Z/BX (Llamas, 2025).	16
Figura 1.14-Diagrama de conexión del sensor HC-SR04 y la tarjeta ESP32.	17
Figura 1.15-Diagrama de conexión del sensor HC-SR04 y la tarjeta ESP32 con una carga de 127 V.	19
Figura 1.16-Instalación de la librería NewPing para el ultrasónico.	19
Figura 1.17-Sensor PIR hc-sr501 conectado a la tarjeta ESP32.	21
Figura 1.18-Sensor Inductivo LJ12A3-4-Z/BX conectado a la tarjeta ESP32.	22
Figura 2.1-Modulo I2C para una pantalla LCD.	26
Figura 2.2-Puertos con soporte para el protocolo I2C de la tarjeta ESP32.	27
Figura 2.3-Pantalla LCD 16X2.	27
Figura 2.4-Terminales de la Pantalla LCD 16X2.....	28
Figura 2.5-Localidades de memoria de la Pantalla LCD 16X2.	29
Figura 2.6- Mostrando caracteres en pantalla LCD.	29
Figura 2.7- Mostrando caracteres en pantalla LCD.	30
Figura 2.8- Pixeles de la Pantalla LCD.	30

Figura 2.9- Creando arrays para dibujar gráficos en la pantalla.	31
Figura 2.10- Memoria CGRAM de la pantalla LCD 16X2 (NAYLAMP MECATRONICS, 2025)	31
Figura 2.11- Conexión de la pantalla LCD a la ESP32.....	32
Figura 3.1-Estructura de una pantalla OLED	38
Figura 3.2-Pantalla OLED 128x64.....	39
Figura 3.3-Filas de la pantalla OLED de 128 pixeles de ancho por 64 pixeles de alto.....	40
Figura 3.4-Columnas de una fila de la pantalla OLED.	40
Figura 3.5-Librería de Adafruit para el funcionamiento de la pantalla OLED.	41
Figura 3.6-Impresión en la posición (0,0).	42
Figura 3.7-Impresión en la posición (64,0).	42
Figura 3.8-Impresión en la posición (6,8).	43
Figura 3.9-Impresión en la posición (0,12), en una pantalla con filas de color diferente.	43
Figura 3.10-Fuente de tamaño 1.....	44
Figura 3.11-Fuente de tamaño 2.....	44
Figura 3.12-Fuente de tamaño 3.....	45
Figura 3.13-Fuente con configuración “White”.	46
Figura 3.14-Fuente con configuración “Black”.	46
Figura 3.15-Fuente con configuración “Black-White”.	46
Figura 3.16-Movimiento de texto usando el comando display.startscrollright/left.	47
Figura 3.17-Conexión de la Pantalla OLED a la ESP32.	48
Figura 3.18-Imagen de Goku para mostrar en la pantalla OLED.	52
Figura 3.19-Seleccionar imagen para mostrar en la pantalla OLED.....	52
Figura 3.20-Parámetros de configuración de la aplicación web.	53
Figura 3.21-Vista previa de la imagen seleccionada.	53
Figura 3.22-Código generado por la aplicación.	54
Figura 3.23-Imagen convertida en un mapa de bits mostrada en la pantalla OLED.....	58
Figura 4.1-Protocolo SPI (APRENDIENDO ARDUINO, 2025).	62
Figura 4.2-Pines con soporte para protocolo SPI.....	62
Figura 4.3-Conexión de la ESP32 al modulo lector de memorias SD.	63
Figura 4.4-Conexión de la ESP32 al sensor de temperatura digital DS18B20	66
Figura 4.5-Circuito para el almacenamiento de datos de temperatura en la tarjeta SD.	67
Figura 4.6-Archivo almacenado en tarjeta SD	69
Figura 4.7-Configuración de Excel para abrir el archivo almacenado.....	70
Figura 4.8-Selección de separación por comas para formar las celdas.	70
Figura 4.9-Finalización del asistente para mostrar información en Excel.	71
Figura 4.10-Información mostrada en Excel.....	71
Figura 5.1-Motorreductor Industrial (México, 2025).....	75
Figura 5.2-Motorreductor para aplicaciones con Arduino y ESP32.	76
Figura 5.3-Cambio de dirección en un motor de CD.	76

Figura 5.4- Configuración de un Puente H para control de motor de CD.....	76
Figura 5.5- Inversión de giro de un motor de CD.	77
Figura 5.5- Circuito Integrado L298N (ALLDATASHEET, 2025).	77
Figura 5.6- Modulo puente H.	78
Figura 5.7- Terminales y voltajes de alimentación del módulo Puente H (Electronilab, 2025).	79
Figura 5.8- Conexión de la ESP32 al módulo puente H L298N.	80
Figura 5.9- Conexión de la ESP32 al módulo puente H L298N para el control de velocidad de un motor.	81
Figura 6.1- Estructura de una URL.	86
Figura 6.2- Esquema cliente-servidor	86
Figura 6.3- Envío de mensajes entre cliente-servidor	87
Figura 6.4- Ejemplo de un mensaje HTTP programado en Arduino IDE.....	87
Figura 6.5- Sensor DHT11 en dos versiones comerciales.	88
Figura 6.6- Librería DHT11 de Adafruit.....	88
Figura 6.6- Control de un LED a través de una conexión WiFi.....	89
Figura 6.7- Circuito para el monitoreo de temperatura y humedad a través de una red WiFi.	91

ÍNDICE DE TABLAS

Tabla 1.1- Características técnicas del sensor ultrasónicos HC-SR04.....	10
Tabla 1.2- Características técnicas del sensor inductivo LJ12A3-4-Z/BX	16

INTRODUCCIÓN

En este documento se presenta un manual de prácticas para las asignaturas “ACP-137 Introducción a IoT” y “AFDE-PPR-23 Principios de IoT”, específicamente en la unidad II del programa de asignatura “Elementos de un sistema IoT”. Este manual incluye diversas actividades guiadas cuyo objetivo es reforzar los conocimientos del estudiantado de las carreras de Ingeniería en Sistemas de Energía e Ingeniería en Redes sobre el uso de sensores y actuadores aplicables en proyectos de Internet de las Cosas (IoT).

La Práctica 1 está enfocada en el reconocimiento y uso de sensores que miden distancia, movimiento y proximidad. La Práctica 2 aborda el uso de una pantalla LCD 16x2 como dispositivo visual en aplicaciones IoT. En la Práctica 3 se desarrollan circuitos utilizando una pantalla OLED, la cual ofrece mayor resolución y funcionalidad que la LCD. La Práctica 4 está orientada al almacenamiento de datos provenientes de sensores en memorias SD.

La Práctica 5 se centra en el control del sentido de giro y la velocidad de motores de corriente directa. Finalmente, la Práctica 6 introduce el uso del módulo ESP32 como servidor web, permitiendo acceder desde un navegador para visualizar datos de sensores, específicamente de temperatura y humedad.

Este manual está diseñado para apoyar al profesorado en la enseñanza de la asignatura, contribuyendo así a la formación profesional del estudiantado del programa educativo de Ingeniería en Redes.

Práctica Guiada 1. Sensores digitales: movimiento y proximidad.



Objetivos:

- Implementar circuitos que operen con sensores ultrasónicos, PIR e inductivos para analizar su funcionamiento, aplicaciones y la programación para su funcionamiento con la Placa de desarrollo ESP32.

Integrantes del equipo:

1.1-Marco Teórico

1.1.1 Sensor ultrasónico HC-SR04

El sensor HC-SR04 de la figura 1.1, es un sensor de distancia de bajo costo que utiliza ultrasonido para determinar la distancia de un objeto. Destaca por su pequeño tamaño, bajo consumo energético, buena precisión y excelente precio.

El sensor HC-SR04 es el más utilizado dentro de los sensores de tipo ultrasonido, principalmente por la cantidad de información y proyectos disponibles en la web. De igual forma es el más empleado en proyectos de robótica como robots laberinto o sumo, y en proyectos de automatización como sistemas de medición de nivel o distancia. (MECATRONICS, 2025)



Figura 1.1-Funcionamiento del sensor HC-SR04

Funcionamiento general

El sensor ultrasónico HC-SR04 envía una señal a una frecuencia de 40 kHz, el oído humano normal escucha en un rango de 20 Hz a 20 kHz. La señal ultrasónica viaja a una velocidad de 344 m/s en el aire. El sensor mide el tiempo en que se envió la señal y esta tarda en regresar (el eco). Similar al método que usan los murciélagos para saber dónde se encuentran los objetos, ya que ellos son prácticamente ciegos, Figura 1.2.

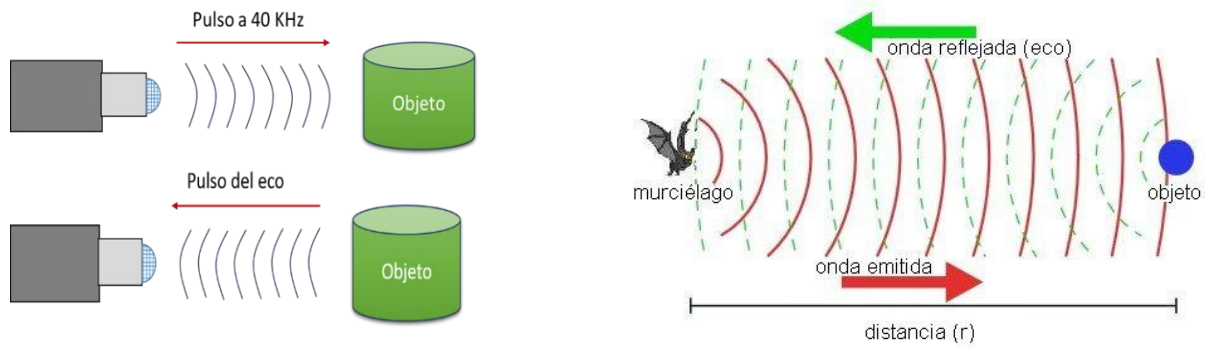


Figura 1.2-Funcionamiento del sensor HC-SR04

Funcionamiento técnico

- El sensor manda una señal de arranque en el pin trigger (disparo) del SR04 y después lee el ancho del impulso que nos proporciona el pin de eco (eco). El pulso de disparo debe tener un ancho mínimo de 10 μ s (por eso la velocidad se expresa en microsegundos).
- Después leemos el pulso de salida de eco y medimos su longitud que es proporcional al eco recibido. En caso de que no se produzca ningún eco, porque no se encuentra un objeto, el pulso de eco tiene una longitud aproximada de 36 ms.
- Hay que dejar un retardo de 10 ms desde que se hace una lectura hasta que se realiza la siguiente, con el fin de que el circuito se estabilice (Morgan, 2014). Lo descrito se muestra en la Figura 1.3.

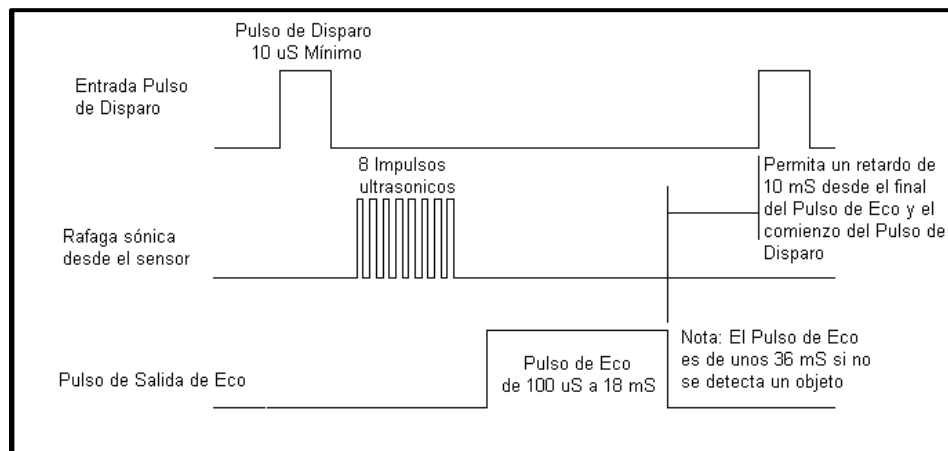


Figura 1.3-Diagrama de pulsos del sensor HC-SR04 (Morgan, 2014)

Para la programación el fabricante recomienda las fórmulas mostradas en la Figura 1.4 para hacer la conversión del ancho del pulso obtenido del sensor a unidades de distancia sean (cm o pies).

$$d = \frac{\text{duración del pulso}}{58} \longrightarrow \text{Fórmula para obtener los valores en cm}$$

$$d = \frac{\text{duración del pulso}}{148} \longrightarrow \text{Fórmula para obtener los valores en Inch (pulgadas)}$$

Figura 1.4-Diagrama de pulsos del sensor HC-SR04

Terminales

- **Vcc:** Voltaje de alimentación de 3.3 a 5 V
- **Trig:** Terminal de disparo (se envía un pulso de 10 μ s para que empiece a enviar el ultrasonido. (Se configura en Arduino como OUTPUT).
- **Echo:** Terminal que regresa un pulso proporcional al tiempo medido. (Se configura en Arduino como OUTPUT).
- **Gnd:** Groud o tierra del sensor.

Características técnicas

Las características de la Tabla 1.1 deben ser consideradas para su programación y para el diseño de alguna base que funcione para el soporte del sensor ultrasónico.

Tabla 1.1- Características técnicas del sensor ultrasónicos HC-SR04

Características técnicas de SRF04	
Rango	2 cm a 4 m
Alimentación	5 V a 30 mA
Frecuencia de señal emitida	40 KHz
Pulso de disparo	10 μ s min. TTL
Puso del Eco	100 μ s – 18 ms
Retardo entre pulsos	10 ms mínimo.
Angulo de medición	A 15° más eficaz, pero puede medir hasta 30°
Peso	10 gr
Tamaño	43 mm x 20 mm x 17 mm

1.1.2 Sensor PIR Hc-sr501

Todos los seres vivos e incluso los objetos emiten radiación electromagnética infrarroja, debido a la temperatura a la que se encuentran, como se muestra en la Figura 1.5.

A mayor temperatura, la radiación aumenta. Esta característica ha dado lugar al diseño de sensores de infrarrojo pasivos, en una longitud de onda alrededor de los 9.4 micrones, los cuales permiten la detección de movimiento, típicamente de seres humanos o animales.

Estos sensores son conocidos como PIR, y toman su nombre de 'Pyroelectric Infrared' ó 'Passive Infrared'.

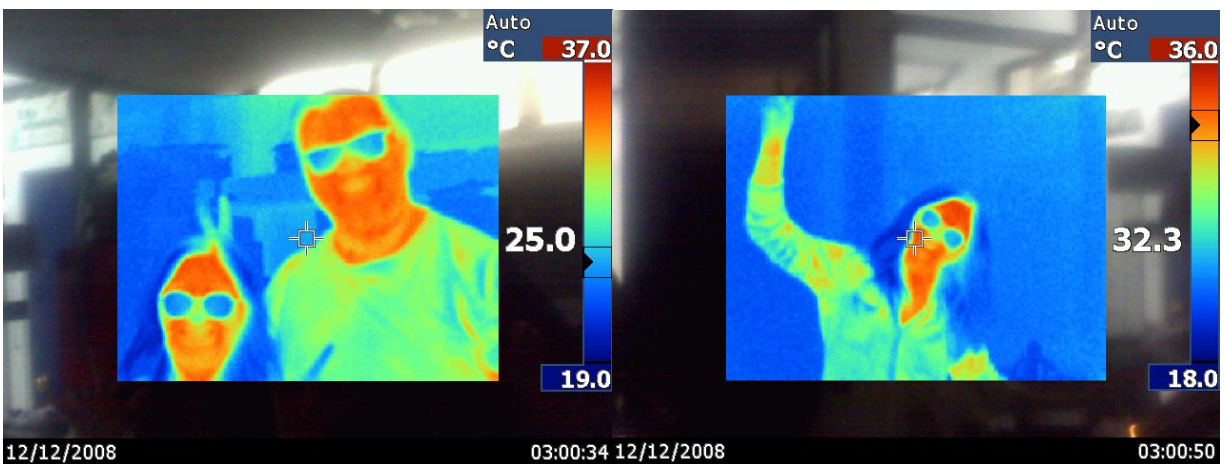


Figura 1.5-Imágenes infrarrojas obtenidas con cámara termográfica.

El lente de Fresnel

El lente de Fresnel de la Figura 1.6 es un encapsulado semiesférico hecho de polietileno de alta densidad cuyo objetivo es permitir el paso de la radiación infrarroja en el rango de los 8 y 14 micrones.

El lente detecta radiación en un ángulo con apertura de 110° y, adicionalmente, concentra la energía en la superficie de detección del sensor PIR, permitiendo una mayor sensibilidad del dispositivo. (FLOTANTE, 2025)

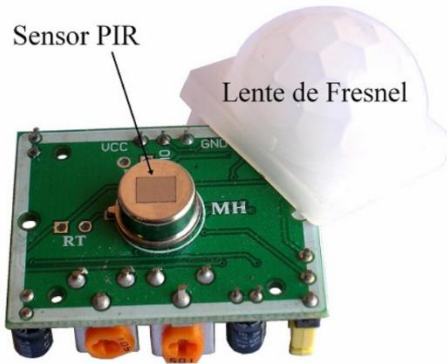


Figura 1.6-El lente Fresnel (FLOTANTE, 2025).

Funcionamiento

Cuando una persona o animal pasa por el rango de detección, primero intercepta una media del sensor, provocando una diferencia de radiación, que a la vez produce una diferencia de potencial o voltaje positivo.

Cuando la persona cruza la otra media del sensor se vuelve a generar una diferencia de radiación, pero negativa, lo que provoca de igual manera un voltaje negativo.

Estas diferencias de radiación generan diferencias de voltaje que es interpretada por un microcontrolador integrado al sensor, detectando el movimiento, como se muestra en la Figura 1.7.

El módulo del sensor PIR HC-SR501 ya está diseñado para devolver un 1 lógico cuando detecta movimiento y un 0 lógico en caso contrario, por lo que tenemos un sensor digital.

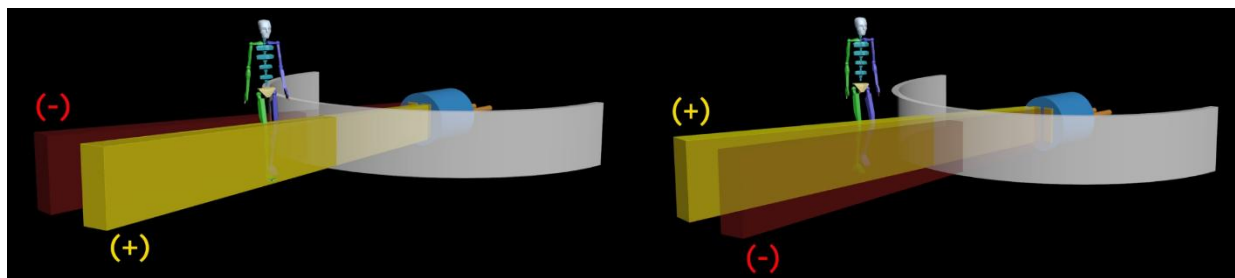


Figura 1.7-Modo de detección del sensor PIR

Rango de detección

El rango de detección de movimiento de los PIR es ajustable y generalmente funcionan con alcances de hasta 7 metros, y con aperturas de 90° a 110° , como se muestra en la Figura 1.8. El montaje del PIR puede realizarse tanto en piso, muro o techo, según convenga a la aplicación.

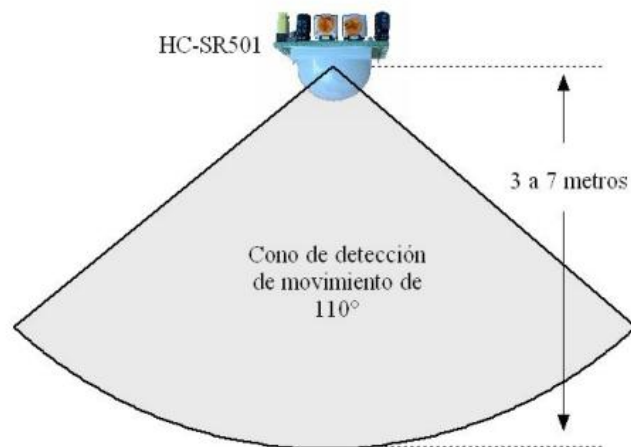


Figura 1.8-Rango de detección del sensor PIR HC-SR501 (FLOTANTE, 2025).

Potenciómetros de ajuste

De acuerdo con la Figura 1.9, el usuario puede ajustar tanto el tiempo de disparo de la señal de alarma de movimiento, como la distancia de detección. Los potenciómetros correspondientes deben girarse en la dirección mostrada para realizar los ajustes.

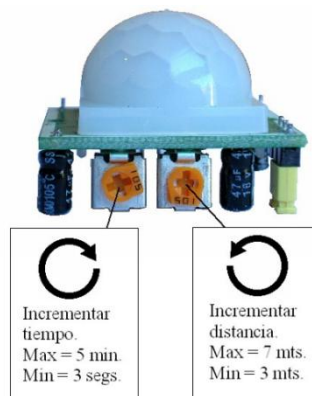


Figura 1.9-Potenciometros de ajuste del sensor PIR HC-SR501 (FLOTANTE, 2025).

Terminales y detalles técnicos

Sus especificaciones técnicas son:

- Usa el PIR LHI778 y el controlador BISS0001
- Voltaje de alimentación: de 5 a 12 VDC
- Consumo promedio: <1 miliampere
- Rango de distancia de 3 a 7 metros ajustable
- Angulo de detección: cono de 110°
- Ajustes: 2 potenciómetros para ajuste de rango de detección y tiempo de alarma activa.
- Jumper para configurar la salida de alarma en modo mono-disparo ó disparo repetitivo (“retriggerable”)
- Salida de alarma de movimiento con ajuste de tiempo entre 3 segundos a 5 minutos.
- Salida de alarma activa Vo con nivel alto de 3.3 volts y 5 mA source, lista para conexión de un led, ó un transistor y relevador.
- Tiempo de inicialización: después de alimentar el módulo HC-SR05, debe transcurrir 1 minuto antes de que inicie su operación normal. Durante ese tiempo, es posible que el módulo active 2 ó 3 veces su salida.
- Tiempo de salida inactiva: cada vez que la salida pase de activa a inactiva, permanecerá en ese estado los siguientes 3 segundos. Cualquier evento que ocurra durante ese lapso es ignorado.
- Temperatura de operación: -15° a +70° C.
- Dimensiones: 3.2 x 2.4 x 1.8 cms

En la figura 1.10 se muestran los elementos del sensor PIR HC-SR01.

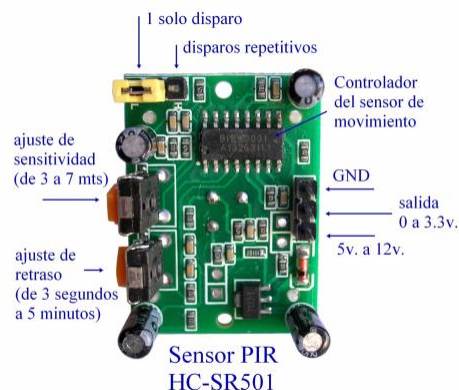


Figura 1.10-Elementos del sensor PIR HC-SR501 (FLOTANTE, 2025).

1.1.3 Sensor inductivo LJ12A3-4-Z/BX

Estos dispositivos permiten a los equipos indicadores o controladores de procesos detectar la presencia de un objeto metálico que se encuentre dentro del rango y dirección de censado.

A diferencia de los sensores capacitivos, los sensores inductivos (Figura 1.11) requieren la presencia de un objeto metálico, pues su principio de funcionamiento es la variación de frecuencia en el campo magnético que emite. Para detección de objetos opacos no metálicos lo indicado es un sensor capacitivo.



Figura 1.11-Sensor Inductivo LJ12A3-4-Z/BX

Funcionamiento

Internamente estos sensores disponen de un generador de campo magnético y una bobina inductora que detecta el campo generado por el propio sensor.

La presencia de un objeto metálico modifica el campo inducido. Al tener una menor resistencia magnética (reluctancia) el campo magnético se "alarga", incrementando la corriente inducida en la bobina sensora (Llamas, 2025), ver Figura 1.12.

Este campo es detectado por la electrónica del sensor, cuya salida se activa cuando un metal es detectado. Como el sensor tiene dos estados, en ocasiones a estos dispositivos se les denomina interruptores inductivos de proximidad.

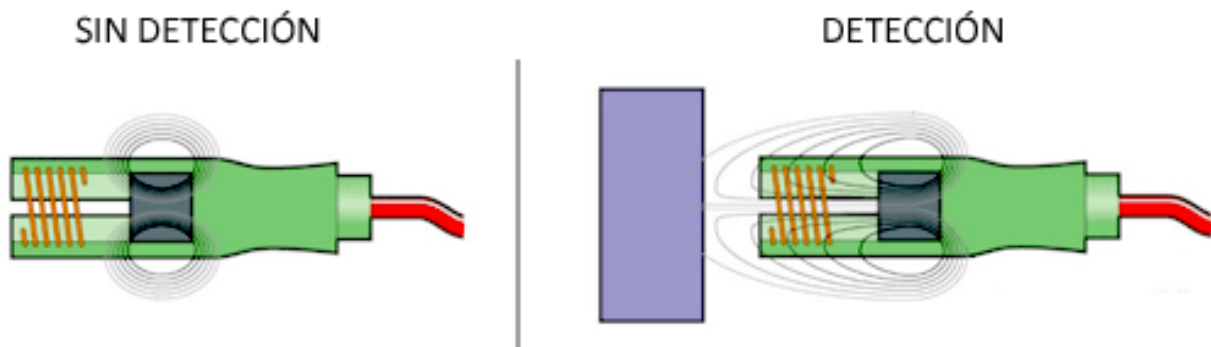


Figura 1.12-Sensor Inductivo LJ12A3-4-Z/BX (Llamas, 2025).

Datos técnicos y terminales

En la figura 1.13 se pueden observar las terminales para la respectiva conexión del sensor.

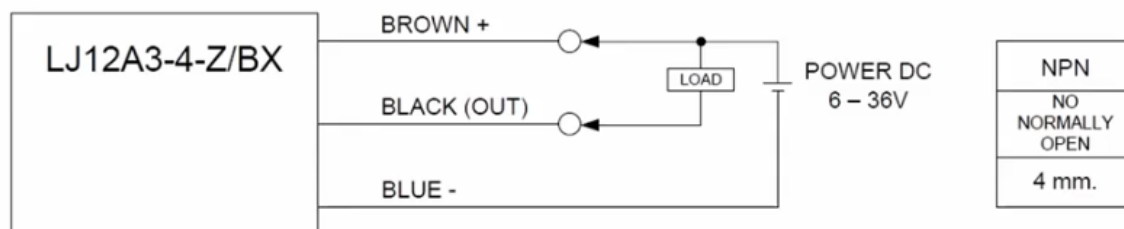


Figura 1.13-Conexión del sensor Inductivo LJ12A3-4-Z/BX (Llamas, 2025).

En la Tabla 1.2 se pueden observar las características técnicas del sensor Inductivo.

Tabla 1.2- Características técnicas del sensor inductivo LJ12A3-4-Z/BX

Características técnicas de SRF04	
Salida	NPN, 3 cables, NO (Normalmente abierto).
Tensión de operación	6 VCD- 36 VCD
Corriente de salida	300 mA
Dimensiones	Diámetro de 12 mm y longitud del cuerpo 64 mm
Largo del cable	100 cm
Rango de detección	4 mm
Detección de objetos	Metales como: Cobre, acero, aluminio, etc.

1.2 Desarrollo de la práctica Guiada I

1.2.1 Material por utilizar.

- 1 Protoboard
- 1 tarjeta ESP32
- Cables de conexión.
- 1 Módulo relevador de estado solido
- 1 sensor ultrasónico HC-SR04
- 1 Sensor inductivo LJ12A3-4-Z/BX
- Sensor De Movimiento Pir Hc-sr501
- 1 resistencia de 150 K Ω
- 1 resistencia de 100 k Ω

1.2.2 Ejercicio 1. Sensor ultrasónico HC-SR04.

a) Realiza la conexión entre los componentes como se muestra en la figura 1.14.

Toma foto del circuito realizado como evidencia para posteriormente agregarlo al reporte.

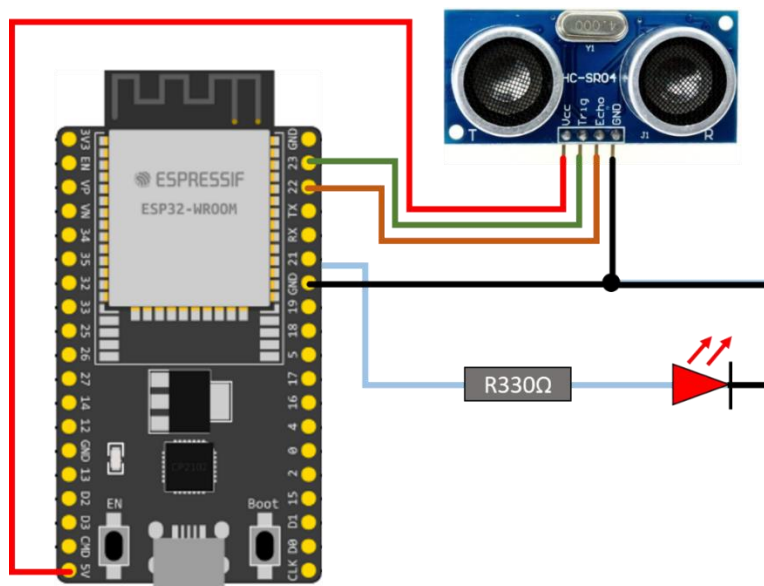


Figura 1.14-Diagrama de conexión del sensor HC-SR04 y la tarjeta ESP32.

- b) Captura el siguiente código y cárgalo a la tarjeta Nodemcu ESP32. Analiza su funcionamiento.

```
#define trig 23
#define echo 22
#define led 21
int duracion;
int distancia;

void setup() {
  Serial.begin (9600);
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);
  pinMode(led,OUTPUT);
}

void loop() {

  digitalWrite(trig, LOW); //Inicio el disparo o trigger apagado para evitar que se
  active solo.
  delayMicroseconds(2); //espero un tiempo antes de activar el trigger para que se
  estabilice el sensor.
  digitalWrite(trig,HIGH); //activo el trigger con un pulso alto
  delayMicroseconds(10); //el pulso para la activación del trigger debe ser mínimo de 10
  us según el fabricante.
  digitalWrite(trig,LOW); //Apago el pulso de disparo
  //Procedo a escuchar el pulso del echo, provocado por el rebote del sonido
  duracion=pulseIn(echo, HIGH); //El comando pulseIn devuelve un valor de tiempo que dura
  el pulso alto del echo

  distancia= (duracion/58); //Para obtener el valor de la distancia el fabricante indica
  dividir entre 58 para convertir a CM
  Serial.print(distancia); //Imprimo en el monitor serial el valor de la distancia.
  Serial.println(" cm"); //Imprimimos una frase
  delay(500); //El fabricante indica que hay que esperar un tiempo de 10 ms, en este caso
  dejamos más tiempo.
  //Líneas de código para encender el LED
  if (distancia <=30 && distancia >=0){ //Si la distancia entre 0 y 30 cm entonces
  digitalWrite(led,HIGH); //Enciendo el Led.
  delay(distancia*5); //Mientras menos sea la distancia el valor del delay será menor.
  Mientras más cerca este el objeto más rápido parpadea.
  digitalWrite(led,LOW); //Apago el Led
  }
}
```

Agrega captura de pantalla del monitor serial donde se observen las mediciones de la distancia medida, agrega el código capturado de igual manera al reporte.

- c) Ahora conectemos un foco de CA de 127 V en lugar del LED, como se muestra en la Figura 1.15:

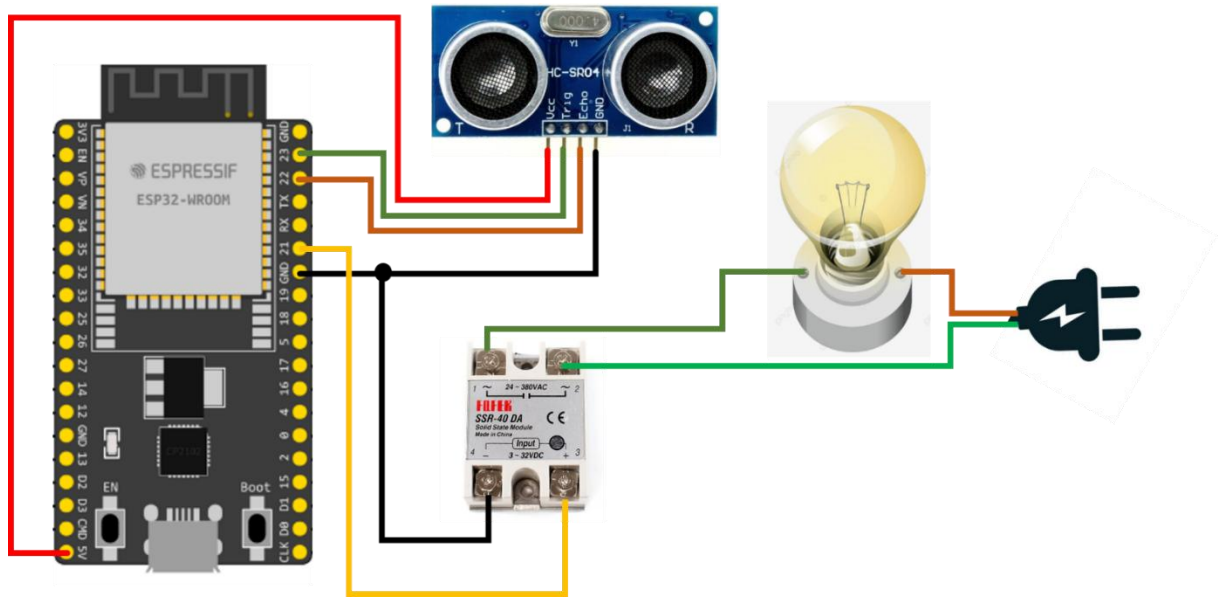


Figura 1.15-Diagrama de conexión del sensor HC-SR04 y la tarjeta ESP32 con una carga de 127 V.

Toma foto del circuito realizado como evidencia para posteriormente agregarlo al reporte.

- d) Instala la librería NewPing, como se muestra en la Figura 1.16.

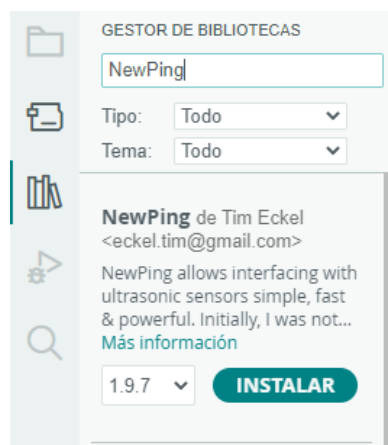


Figura 1.16-Instalación de la librería NewPing para el ultrasónico.

e) Ahora captura el siguiente código, el cual realiza la misma operación de medir distancia, pero de una manera más sencilla.

```
#include <NewPing.h>
#define TRIGGER 23
#define ECHO 22
#define MAX_DISTANCIA 200
#define led 21
NewPing sonar(TRIGGER, ECHO, MAX_DISTANCIA);

int distancia;

void setup() {
  Serial.begin(9600);
  pinMode(led,OUTPUT);
}
void loop() {
  delay(1000); // esperar 50ms entre pings (29ms como minimo)
  Serial.print(sonar.ping_cm()); // obtener el valor en cm (0 = fuera de rango)
  Serial.println("cm");

  unsigned int distancia=sonar.ping_cm();
  distancia=sonar.ping_cm();

  if (distancia<=30){
    digitalWrite(led,HIGH);
  }else{
    digitalWrite(led,LOW);
  }
}
```

Agrega captura de pantalla del monitor serial donde se observen las mediciones de la distancia medida, agrega el código capturado de igual manera al reporte.

1.2.3 Ejercicio 2. Sensor de movimiento PIR Hc-sr501

a) Realiza la conexión entre los componentes como se muestra en la Figura 1.17.

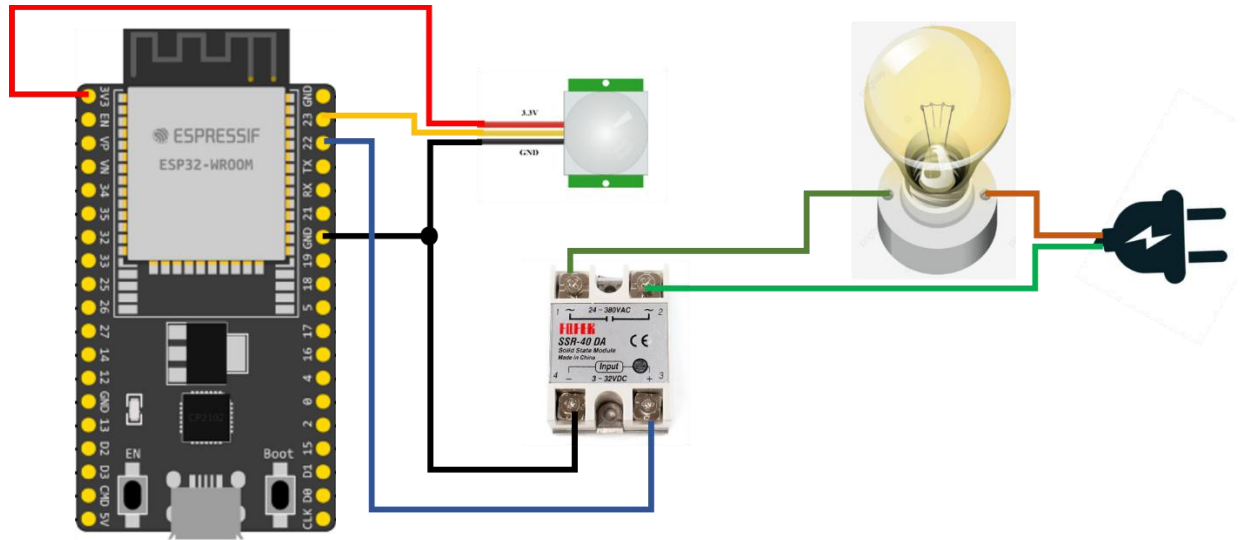


Figura 1.17-Sensor PIR hc-sr501 conectado a la tarjeta ESP32.

b) Captura y analiza el siguiente código

```
#define PIR 23
#define rele 22
int estado=0;

void setup() {
  Serial.begin(9600);
  pinMode(PIR, INPUT); //Configuramos el puerto 2 digital como entrada (Lee la
  señal del sensor)
  pinMode(rele,OUTPUT); //Configuramos el puerto 3 digital como salida (envía la
  señal al relevador)
  delay (5000); //Esperamos 5 segundos para esperar que se estabilicé el sensor
}

void loop() {
  estado=digitalRead(PIR); //leemos la señal digital que envía el sensor PIR
  if (estado==1){ //Si se detecta movimiento
    digitalWrite(rele,HIGH); //enciendo el relevador
    Serial.println("Detectando movimiento"); //Imprimo en el monitor serial
    delay (4000); //espero 4 segundos para realizar la próxima lectura
  } else{ //si no hay movimiento
    digitalWrite(rele,LOW); //apago el relevador
  }
}
```

Agrega captura de pantalla del monitor serial donde se observe que el sensor está detectando, agrega el código capturado de igual manera al reporte.

1.2.4 Ejercicio 3. Sensor inductivo LJ12A3-4-Z/BX

a) Realiza la conexión entre los componentes como se muestra en la Figura 1.18.

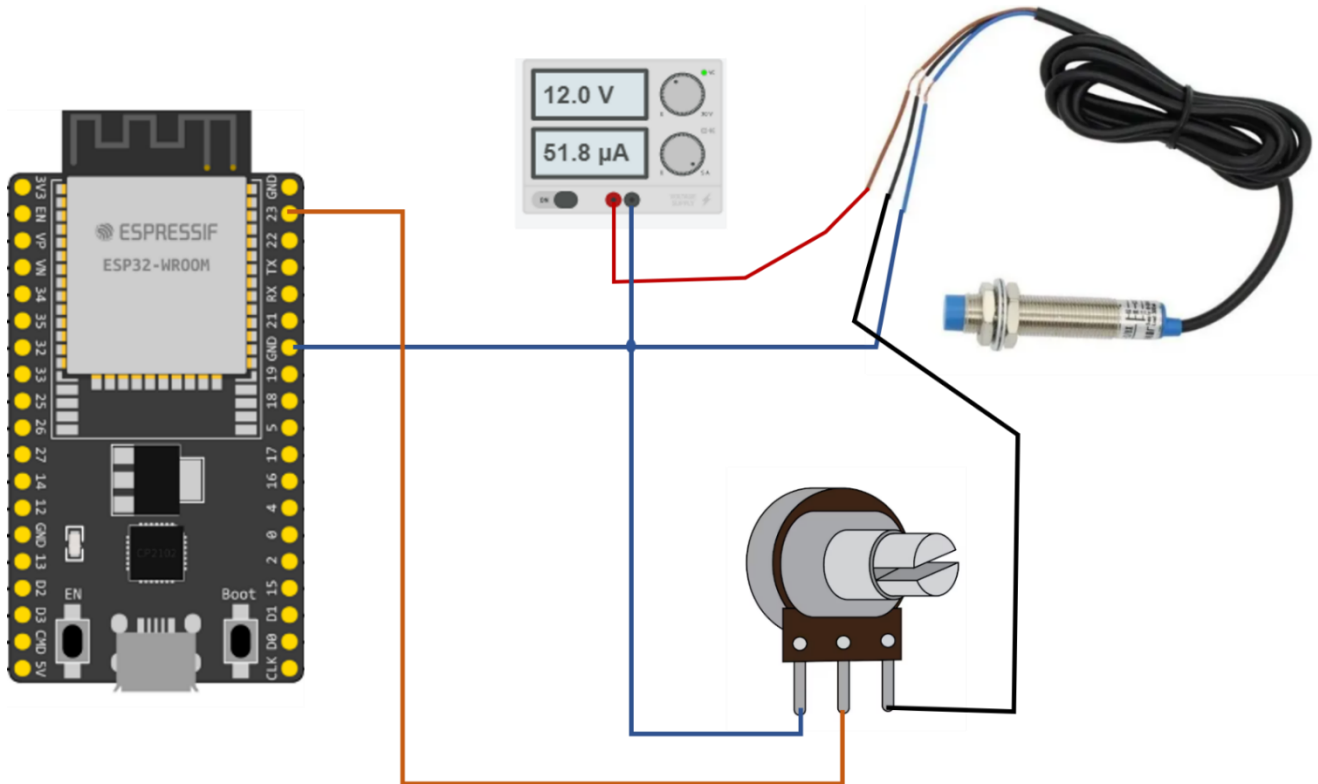


Figura 1.18-Sensor Inductivo LJ12A3-4-Z/BX conectado a la tarjeta ESP32.

b) Captura y analiza el siguiente código.

```
#define sensorInd 23
int estado=0;

void setup() {
  Serial.begin(9600);
  pinMode(sensorInd, INPUT); //Configuramos el puerto 2 digital como entrada (Lee
la señal del sensor)
}
void loop() {
  estado=digitalRead(sensorInd); //leemos la señal digital que envía el sensor
inductivo
  if (estado==0){ //Si se detecta metal
    Serial.println("Detectando metal"); //Imprimo en el monitor serial
    delay (2000); //espero 2 segundos para realizar la proxima lectura
  } else{ //si no hay metales
    Serial.println("*****");
  }
}
```

Condiciones de Entrega

El documento debe incluir los siguientes elementos:

- Portada
- Introducción (media cuartilla)
- Desarrollo del contenido
- Conclusiones

Convierte tu documento a formato **.pdf**, súbelo a la plataforma Microsoft Teams O Moodle. Utiliza el Acrobat 10 o posterior. No le pongas contraseñas ni protecciones a tu archivo.

Guarda todo tu documento con la siguiente estructura ***PACTICA_GUIADA1_Iniciales de los integrantes.***

Escala de Evaluación

Carrera:	Ingeniería en Redes, Ingeniería en Sistemas de Energía e Ingeniería en Inteligencia Artificial.
Asignatura:	Introducción a Internet de las Cosas
Unidad(es) temática(s):	Unidad I
Actividad de Aprendizaje	Práctica Guiada 1. Sensores digitales: movimiento y proximidad

Dimensiones o criterios a evaluar	Puntos por criterio	Puntos obtenidos	Observaciones
Realiza el ejercicio 1. Logra medir la distancia usando el código extenso sin librerías con ayuda del sensor ultrasónico. Agrega toda la evidencia solicitada en el desarrollo de la actividad, como capturas de pantalla y fotos del circuito.	2		
Realiza el ejercicio 1. Logra medir la distancia usando el código con librerías. Agrega toda la evidencia solicitada en el desarrollo de la actividad, como capturas de pantalla y fotos del circuito.	2		
Realiza el ejercicio 2. Realiza la detección de personas con el sensor PIR y puede ver el estado del sensor en el monitor serial. Agrega toda la evidencia solicitada en el desarrollo de la práctica como captura de pantalla y fotos del circuito.	2		
Realiza el ejercicio 3. Logra detectar metales con el sensor inductivo. Observa en el monitor serial que el sensor está detectando metales. Agrega toda la evidencia solicitada en el desarrollo de la práctica como captura de pantalla y fotos del circuito.	2		
El documento cumple con la estructura solicitada, conteniendo todas las partes marcadas.	1		
Realiza una conclusión correcta del tema y de las competencias alcanzadas interrelacionándolos e indica lo aprendido y la manera de lograrlo.	1		
Total de Puntos	10		

*** Nota importante:** En caso de entregas fuera del tiempo marcado se aplicará una penalización de 1 punto menos a la calificación obtenida. Todos los integrantes deben de participar en la práctica o no se considerará en la calificación aquel que no se presente.

Práctica Guiada 2. La pantalla LCD



Objetivos:

- Implementar circuitos que usen la pantalla LCD para aplicaciones de Internet de las Cosas.

Integrantes del equipo:

2.1 Marco Teórico

2.1.1 El módulo I2C

I2C es un bus de comunicaciones en serie. Su nombre viene de Inter-Integrated Circuit (Inter-Circuitos Integrados), su diseñador es Philips.

La velocidad es de 100 kbit/s en el modo estándar, aunque también permite velocidades de 3.4 Mbit/s. Es un bus muy usado en la industria, principalmente para comunicar microcontroladores y sus periféricos en sistemas integrados (Embedded Systems) y generalizando más para comunicar circuitos integrados entre si que normalmente residen en un mismo circuito impreso (APRENDIENDO ARDUINO, 2025), en la Figura 2.1 se observa un módulo I2C para adaptarse a una pantalla LCD.



Figura 2.1-Modulo I2C para una pantalla LCD.

I²C o I2C es un protocolo síncrono. I2C usa solo 2 cables, uno para el reloj (**SCL**) y otro para el dato (**SDA**). Esto significa que el maestro y el esclavo envían datos por el mismo cable, el cual es controlado por el maestro, que crea la señal de reloj. I2C no utiliza selección de esclavo, sino direccionamiento.

- SDA: datos
- SCL: reloj
- GND: tierra
- VCC: Tensión o voltaje de alimentación.

La tarjeta ESP32 dispone de soporte I2C por hardware vinculado físicamente a ciertos pines (22 SCL y 21 SDA), como se muestra en la Figura 2.2. También es posible emplear cualquier otro grupo de pines como bus I2C a través de software, pero en ese caso la velocidad será mucho menor.

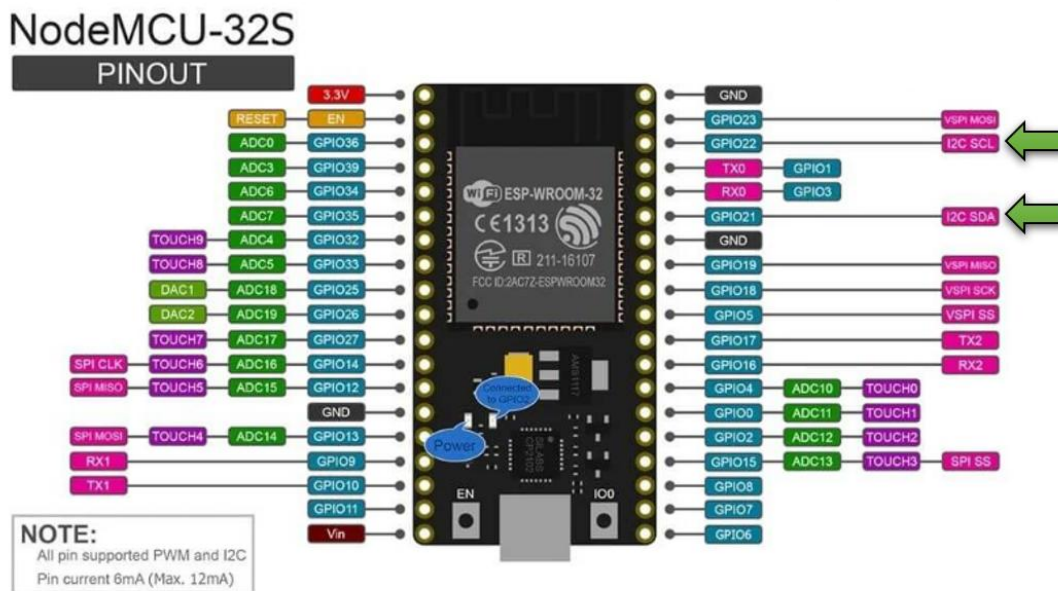


Figura 2.2-Puertos con soporte para el protocolo I2C de la tarjeta ESP32.

La librería para manejar el bus I2C en Arduino es: Wire: <http://arduino.cc/en/reference/wire> Esta librería permite comunicar con I2C/TWI Arduino con otros dispositivos.

2.1.2 La pantalla LCD

La pantalla de cristal líquido o **LCD (Liquid Crystal Display)** es un dispositivo micro-controlado de visualización gráfico para la representación de caracteres, símbolos e incluso dibujos (en algunos modelos).

La pantalla es de 16X2, es decir esta dispone de 2 filas de a 16 caracteres cada una, Figura 2.3. Este dispositivo está gobernado internamente por un microcontrolador que regula todos los parámetros de presentación.



Figura 2.3-Pantalla LCD 16X2.

Terminales de la pantalla LCD

En la figura 2.4 se muestran las terminales de la pantalla LCD sin modulo I2C.

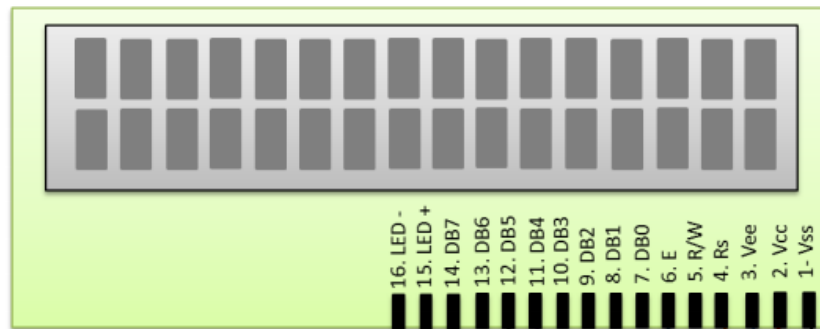


Figura 2.4-Terminales de la Pantalla LCD 16X2.

A continuación, se describen cada una de las terminales:

- **Pin 1 Vss**-Conexión a tierra de la pantalla LCD.
- **Pin 2 Vcc**- Conexión a tensión positiva de la pantalla LCD (5 V).
- **Pin 3 Vee**- Esta terminal se puede mandar directamente a tierra o si se desea se puede conectar a un potenciómetro para el control del contraste de la LCD.
- **Pin 4 RS**- Trabaja paralelamente al Bus de datos del módulo LCD, Pines 7 al 14, es decir, cuando RS es cero, el dato presente en el bus corresponde a un registro de control o instrucción, pero cuando RS es uno, el dato presente en el bus corresponde a un registro de datos o carácter alfanumérico
- **Pin 5 R/W**- (Read/Write), este pin es utilizado para leer un dato desde la pantalla LCD o para escribir un dato en la pantalla LCD. Si R/W = 0, esta condición indica que podemos escribir un dato en la pantalla. Si R/W = 1, esta condición nos permite leer un dato desde la pantalla LCD.
- **Pin 6 E**- (Enable), este es el pin de habilitación, es decir, si E = 0 el módulo LCD se encuentra inhabilitado para recibir datos, pero si E = 1, el módulo LCD se encuentra habilitado para trabajar, de tal manera que podemos escribir o leer desde el modulo LCD.
- **Pin 7-14 DB**- (Bus de datos) El Pin 7 hasta el Pin 14 representan 8 líneas que se utilizan para colocar el dato que representa una instrucción para el modulo LCD o un carácter alfanumérico.
- **Pin 15-16 (LED+ y LED-)** Pin 15-16: "BackLight", en muchos modelos de LCD, los pines 15 y 16 son respectivamente el "Ánodo" y el "Cátodo", aunque se pueden encontrar en el mercado modelos de pantallas LCD donde esta condición es configurable desde la parte posterior del circuito impreso a través de "Jumpers", o

conexiones donde podemos invertir los Pines, de manera tal que el Pin 15 sea el “Cátodo” y el Pin 16 el “Ánodo”.

Localidades de memoria en la Pantalla LCD

Cuando se requiere mostrar un carácter en la pantalla LCD, tenemos que indicar en nuestro algoritmo de programación en que parte de la pantalla se visualizará éste. Cada una de las 16 posiciones de la pantalla tienen una dirección única de memoria (DD RAM), a la cual se puede hacer referencia y así decidir la posición del carácter. En la tabla, se pueden observar las 32 direcciones DD RAM de una pantalla de 16x2, como se muestra en la Figura 2.5.

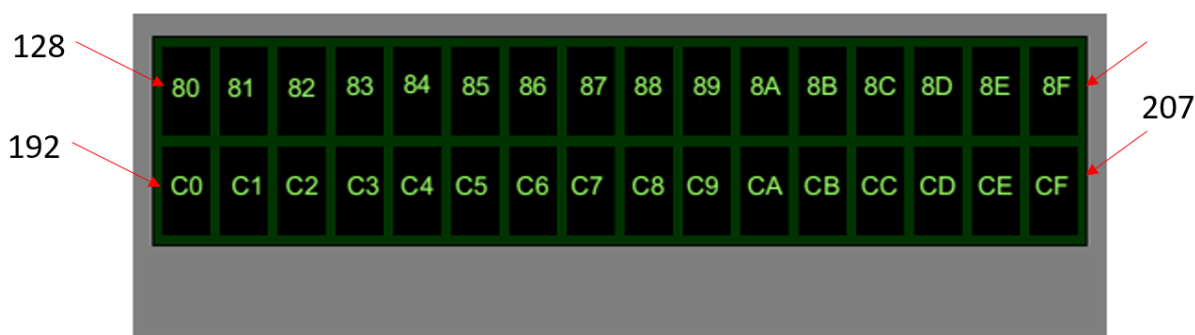


Figura 2.5-Localidades de memoria de la Pantalla LCD 16X2.

Usando la librería “LiquidCrystal-I2C” en el IDE de Arduino basta con decir el número de columna y el número de fila donde debo posicionar el cursor para mostrar los caracteres, como se muestra en la Figura 2.6.

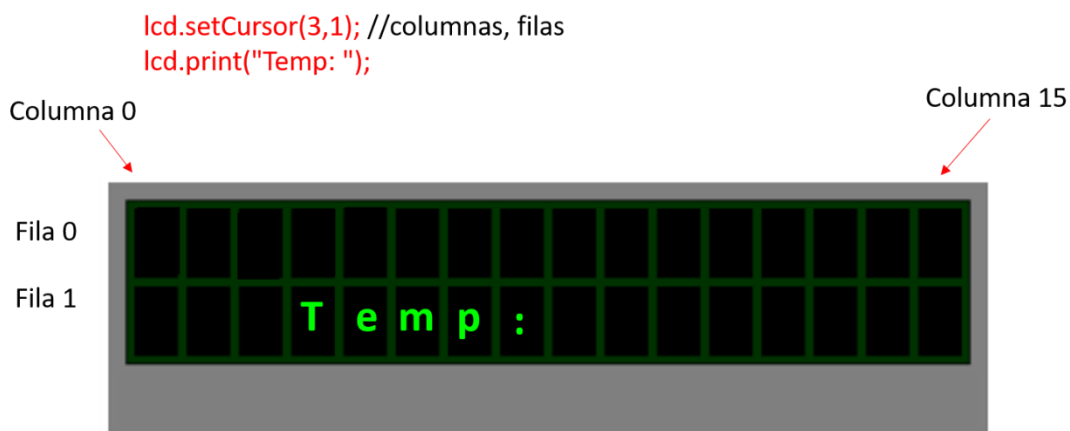


Figura 2.6- Mostrando caracteres en pantalla LCD.

Caracteres en la pantalla LCD

La pantalla LCD posee una memoria llamada CGRAM (Character Generator RAM), destinada para que el usuario pueda almacenar hasta un *máximo de 8 caracteres o figuras* personalizadas.

Estos por si se requiere un carácter que no se encuentra en la memoria de la pantalla almacenada o para realizar alguna animación. Cada carácter es representado como un bloque constituido normalmente por 5 columnas y 8 filas, cada una de las filas de 8 bits (1 byte), es decir, cada carácter se compone de ocho bytes, como se observa en la Figura 2.7.

Nota: Los 3 bits más significativos de cada byte (3 bits más significativos de cada fila), son tomados como ceros.

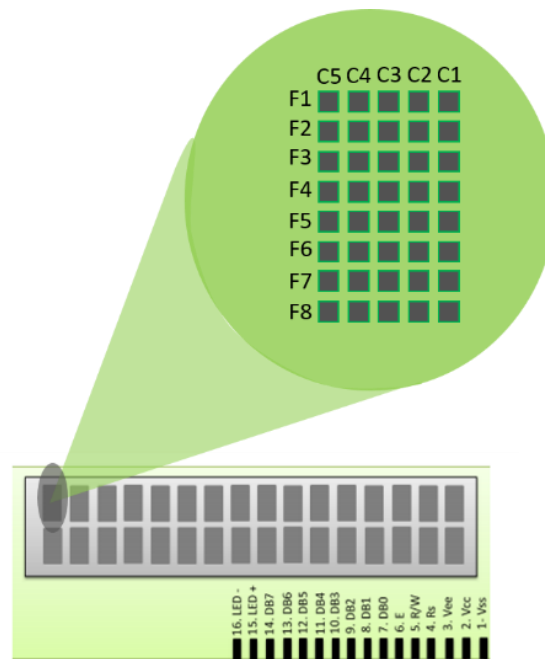


Figura 2.7- Mostrando caracteres en pantalla LCD.

Entonces cada cuadro que contiene la pantalla LCD realmente es un arreglo de pixeles, como se muestra en la Figura 2.8.

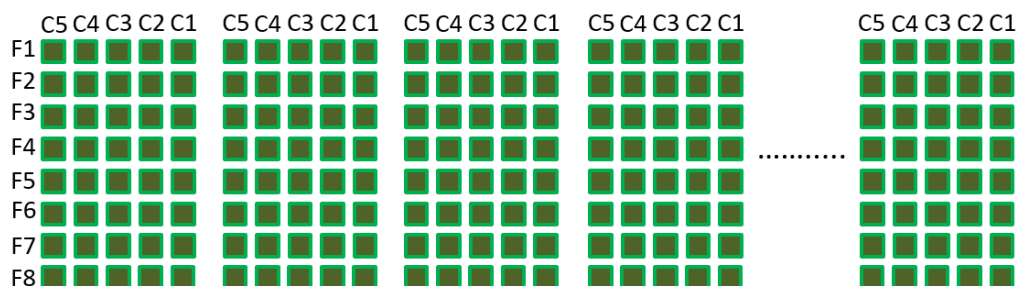


Figura 2.8- Pixeles de la Pantalla LCD.

Los caracteres son creados en una matriz de 8 elementos, usando la librería mencionada. Observe la Figura 2.9, los pixeles que son remarcados se activan con “1” y los que no se agrega un cero dentro del arreglo.

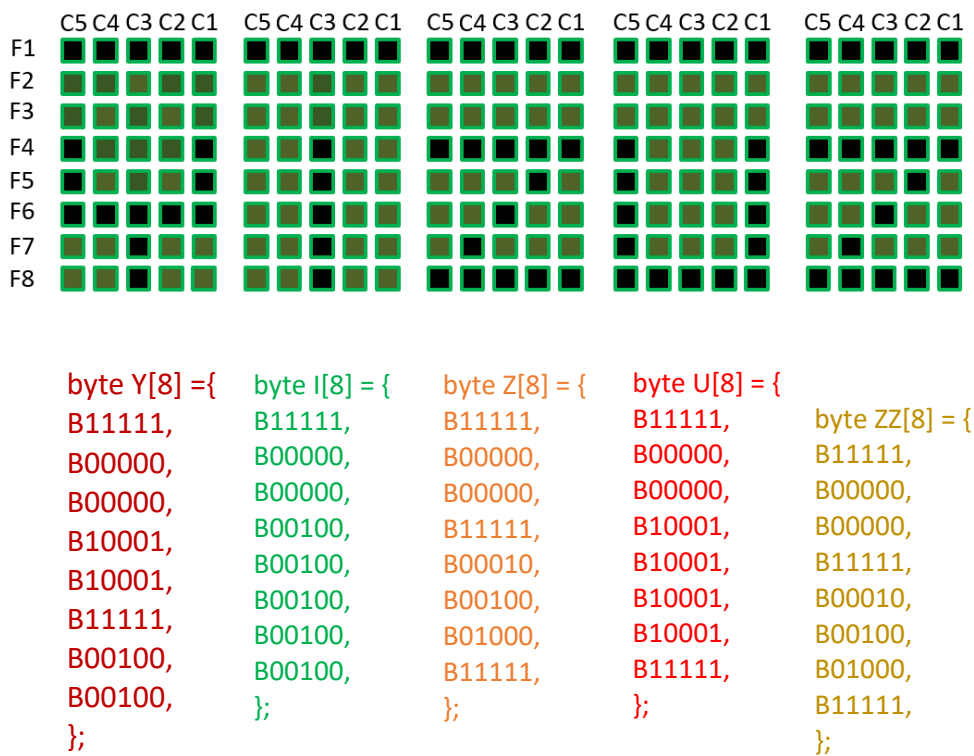


Figura 2.9- Creando arrays para dibujar gráficos en la pantalla.

Una vez creados los arrays, se almacenan en la memoria CGRAM de la pantalla. La memoria CGRAM contiene 8 posiciones de 8 bytes, como se muestra en la Figura 2.10.

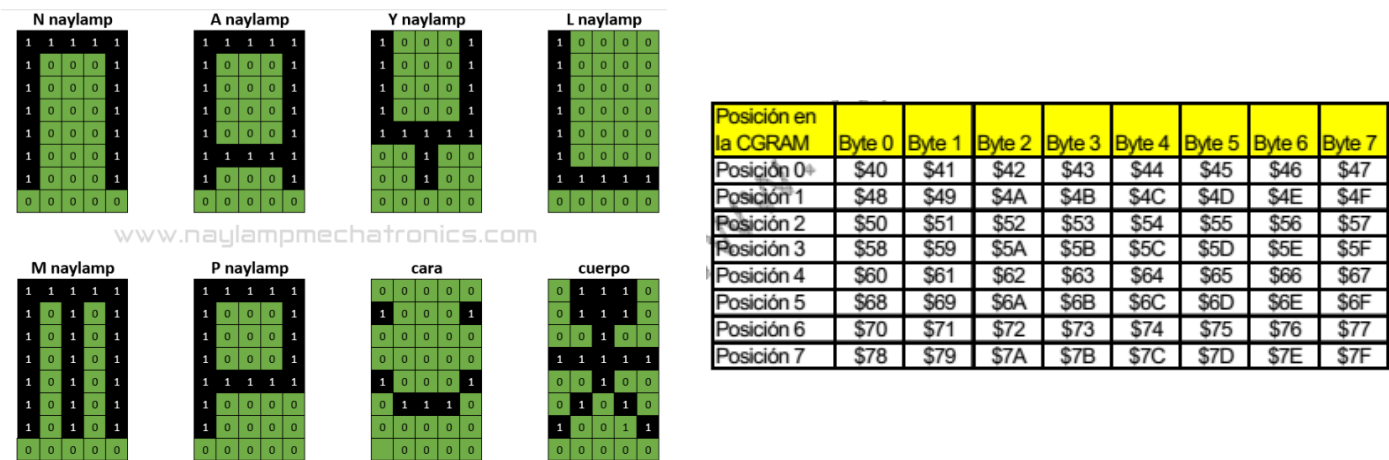


Figura 2.10- Memoria CGRAM de la pantalla LCD 16X2 (NAYLAMP MECATRONICS, 2025) .

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

//Crear el objeto lcd dirección 0x3F y 16 columnas x 2 filas
LiquidCrystal_I2C lcd(0x3F,16,2); //

void setup() {
  // Inicializar el LCD
  lcd.init();

  //Encender la luz de fondo.
  lcd.backlight();
  //lcd.noBacklight();

  // Escribimos el Mensaje en el LCD.
  lcd.setCursor(0,0); //columnas /filas: Inicia desde cero
  lcd.print("Incubadora");
}

void loop() {
```

```

// Ubicamos el cursor en la primera posición(columna:0) de la segunda línea(fila:1)
lcd.setCursor(0, 1);
// Escribimos el número de segundos transcurridos
lcd.print(millis()/1000); //MILLIS DEVUELVE EL VALOR EN MILLISEGUNDOS DESDE QUE SE INCIO LA PLACA
lcd.print(" Segundos");
delay(100);
}

```

c) Agrega una fotografía del texto mostrado en la pantalla LCD.

d) Puedes descargar las librerías de los siguientes enlaces.

Puedes descargar la librería del siguiente enlace:

Librería Wire.h: <http://arduino.cc/en/reference/wire>

Librería LiquidCrystal_I2C.h:

https://codeload.github.com/marcoschwartz/LiquidCrystal_I2C/zip/master

2.2.3 Ejercicio 2. Caracteres en la pantalla LCD

a) Usando la conexión de la pantalla LCD y el arduino del ejercicio 1 implementa el siguiente código.

```

#include <Wire.h>
#include <LiquidCrystal_I2C.h>

//Crear el objeto lcd dirección 0x3F y 16 columnas x 2 filas
LiquidCrystal_I2C lcd(0x3F,16,2); //

byte Y[8] = { //se declara la matriz
B11111, //Se escriben los valores de cada bit que se requieren almacenar anteponiendo la
letra B indicando que se tratan de bits.
B00000,
B00000,
B10001,
B10001,
B11111,
B00100,
B00100,
};
byte I[8] = {
B11111,
B00000,
B00000,
B00100,
B00100,
B00100,
B00100,
B00100,
};

```

```

byte Z[8] = {
  B11111,
  B00000,
  B00000,
  B11111,
  B00010,
  B00100,
  B01000,
  B11111,
};
byte U[8] = {
  B11111,
  B00000,
  B00000,
  B10001,
  B10001,
  B10001,
  B10001,
  B11111,
};

byte cara[8] = {
  B00000,
  B10001,
  B00000,
  B00000,
  B10001,
  B01110,
  B00000,
};
byte cuerpo[8] = {
  B01110,
  B01110,
  B00100,
  B11111,
  B00100,
  B01010,
  B10001,
  B00000,
};

void setup() {
  // Inicializar el LCD
  lcd.init();

  //Encender la luz de fondo.
  lcd.backlight();

  //creamos los nuevos caracteres
  lcd.createChar (0,Y);
  lcd.createChar (1,I);
  lcd.createChar (2,Z);
  lcd.createChar (3,U);
  lcd.createChar (4,cara);
  lcd.createChar (5,cuerpo);

  // Escribimos el texto en el LCD

```

```

lcd.setCursor(3, 0); //COLUMNA Y FILA
lcd.print(" IOT ");
lcd.setCursor(0, 1);
lcd.write (byte (4));
lcd.print(" ");
lcd.write (byte (5));
lcd.print(" ");
lcd.write (byte (0));
lcd.write (byte (1));
lcd.write (byte (2));
lcd.write (byte (3));
lcd.write (byte (2));
lcd.print(" ");
lcd.write (byte (5));
lcd.print(" ");
lcd.write (byte (4));
}

void loop () {}

```

b) Analiza el código y agrega una fotografía de los caracteres mostrados al reporte.

Condiciones de Entrega

El documento debe incluir los siguientes elementos:

- Portada
- Introducción (media cuartilla)
- Desarrollo del contenido
- Conclusiones

Convierte tu documento a formato **.pdf**, súbelo a la plataforma Microsoft Teams O Moodle. Utiliza el Acrobat 10 o posterior. No le pongas contraseñas ni protecciones a tu archivo.

Guarda todo tu documento con la siguiente estructura ***PACTICA_GUIADA2_Iniciales de los integrantes.***

Escala de Evaluación

Carrera:	Ingeniería en Redes, Ingeniería en Sistemas de Energía e Ingeniería en Inteligencia Artificial.
Asignatura:	Introducción a Internet de las Cosas
Unidad(es) temática(s):	Unidad I
Actividad de Aprendizaje	Práctica Guiada 2. La pantalla LCD.

Dimensiones o criterios a evaluar	Puntos por criterio	Puntos obtenidos	Observaciones
Realiza el ejercicio 1. Logra mostrar los caracteres en pantalla. El profesor valida su funcionamiento.	3		
Agrega capturas de pantalla y fotografías como evidencia en el reporte. Del ejercicio 1.	1		
Realiza el ejercicio 2. Muestra imágenes creadas en la pantalla LCD.	3		
Agrega capturas de pantalla y fotografías como evidencia en el reporte. Del ejercicio 2.	1		
El documento cumple con la estructura solicitada, conteniendo todas las partes marcadas.	1		
Realiza una conclusión correcta del tema y de las competencias alcanzadas e indica lo aprendido y la manera de lograrlo.	1		
Total de Puntos	10		

*** Nota importante:** En caso de entregas fuera del tiempo marcado se aplicará una penalización de 1 punto menos a la calificación obtenida. Todos los integrantes deben de participar en la práctica o no se considerará en la calificación aquel que no se presente.

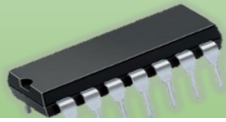
Práctica Guiada 3. La pantalla OLED.



Objetivos:

- Implementar soluciones de IoT que incorporen indicadores visuales como las pantallas OLED, las cuales tienen mayor definición que una pantalla LCD.

Integrantes del equipo:



3.1-Marco Teórico

3.1.1 La pantalla OLED

Las pantallas OLED (Organic Light Emitting Diode, Diodo Orgánico de Emisión de Luz) están disponibles en una variedad de tamaños (como 128×64 y 128×32) y colores (como blanco, azul y bicolor). Algunas tienen una interfaz I²C, mientras que otras tienen una interfaz SPI. Consumen menos energía que una pantalla LCD.

Cada píxel en una pantalla OLED es un diodo orgánico de emisión de luz (OLED) que, al recibir una corriente eléctrica, emite su propia luz. Para mostrar información, cada píxel se controla individualmente, determinando si debe estar encendido o apagado para representar un punto de la imagen o texto. (ChatGPT-GPT-4o, 2025) Su estructura básica se muestra en la Figura 3.1.

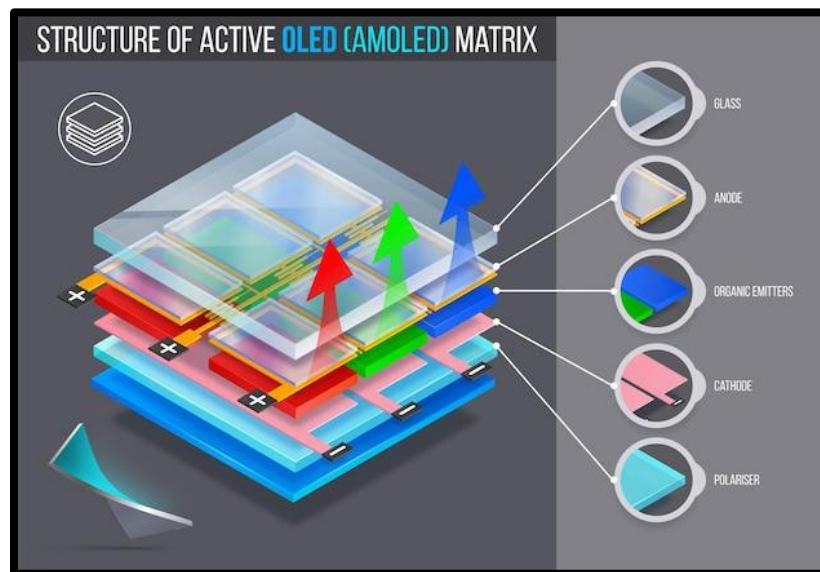


Figura 3.1-Estructura de una pantalla OLED .

Funcionamiento general

La pantalla OLED, mostrada en la Figura 3.2 se destaca por su gran contraste, mínimo consumo de energía y buena calidad de imagen. El display OLED posee una resolución de 128*64 píxeles, permitiendo controlar cada píxel individualmente y mostrar tanto texto como gráficos. La interfaz de comunicación es I2C, lo que le permite trabajar a 5V directamente gracias a su regulador de voltaje en placa, por ende, puede trabajar con sistemas de 3.3V o 5V sin necesidad de conversores externos. El driver que utiliza el módulo es SSD1306, su función es comunicar con el microcontrolador para obtener los datos y enviarlos a la pantalla OLED a través de 4 pines.



Figura 3.2-Pantalla OLED 128x64

La pantalla OLED tiene 8 filas denominadas páginas. Cada fila tiene 8 pixeles de alto por 128 de largo, como se muestra en la Figura 3.3.

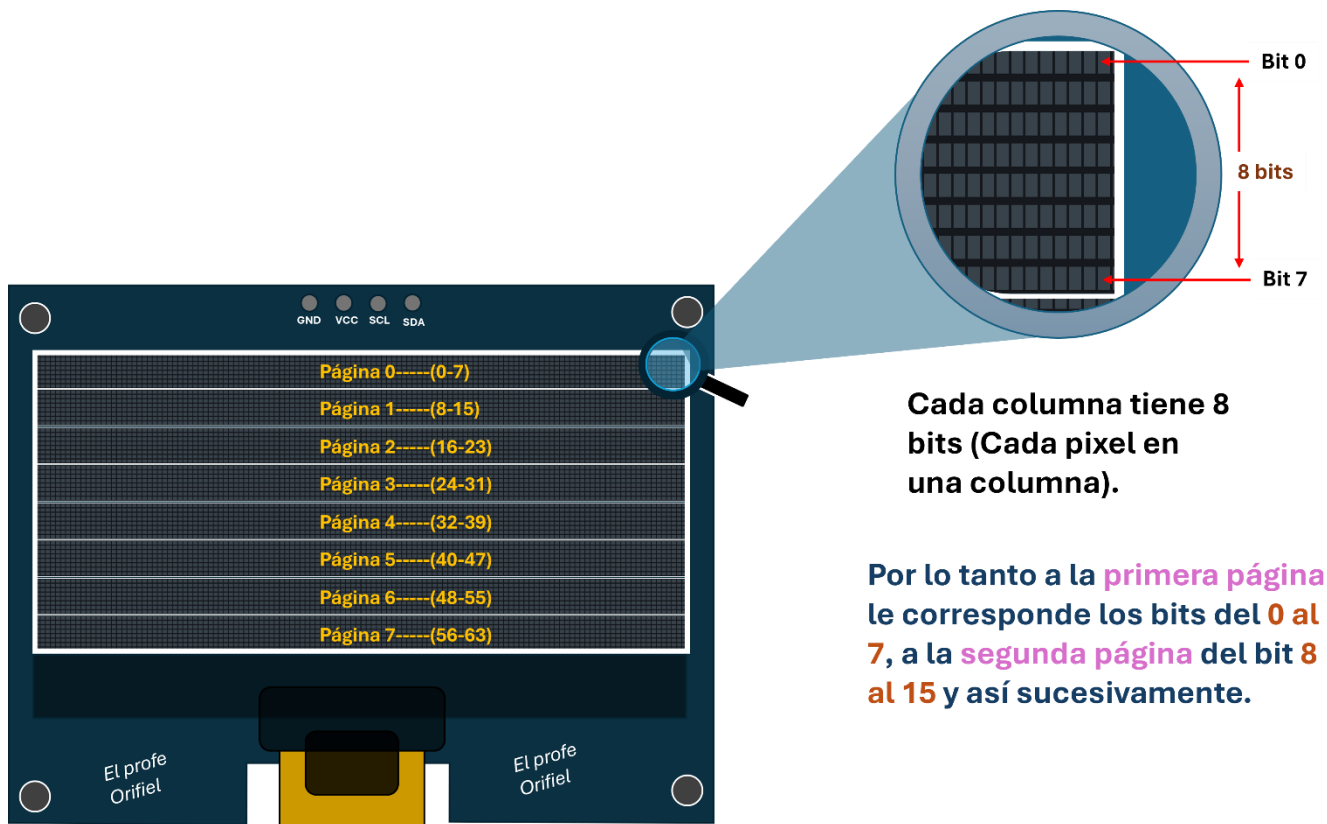


Figura 3.3-Filas de la pantalla OLED de 128 pixeles de ancho por 64 pixeles de alto.

En la Figura 3.4 se puede ver más a detalle una fila de la pantalla la cual tiene 28 columnas de pixeles de ancho.

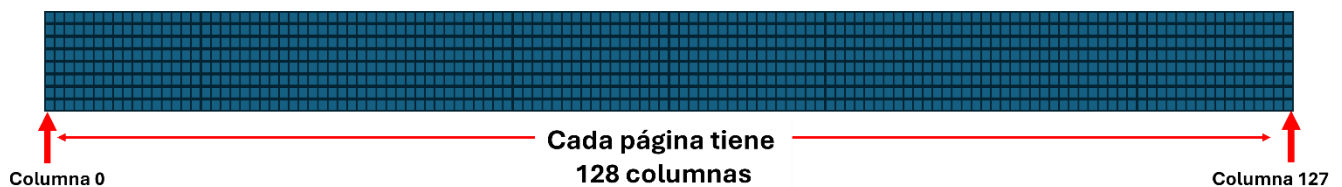


Figura 3.4-Columnas de una fila de la pantalla OLED.

Para poner en funcionamiento esta pantalla se puede usar la librería: `#include <Adafruit_SSD1306.h>`, la cual puedes instalar desde el gestor de librerías del IDE de Arduino, como se muestra en la Figura 3.5.



Figura 3.5-Librería de Adafruit para el funcionamiento de la pantalla OLED.

Imprimir texto en la pantalla.

Usando la librería de **Adafruit** se usa el siguiente comando para ubicar el texto en pantalla:

```
display.setCursor(X, Y);
```

Este comando define el punto de inicio donde se escribirá la palabra.

Puede tomar valores de:

- **X:** De **0 a 127** (horizontal, de izquierda a derecha)
- **Y:** De **0 a 63** (vertical, de arriba abajo).

Por ejemplo, el siguiente comando imprime el texto “**HOLA**” en la pantalla como se muestra en la Figura 3.6.

```
display.setCursor(0,0); //No. De bit de Columna/No. De bit de fila
```

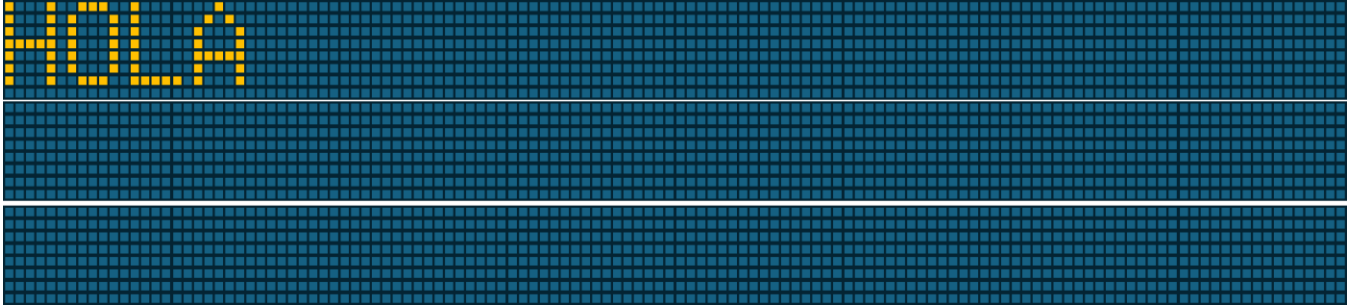


Figura 3.6-Impresión en la posición (0,0).

Por otro lado, el siguiente comando, imprime la palabra “**HOLA**” en la posición mostrada en la Figura 3.7.

```
display.setCursor(64,0); //No. De bit de Columna/No. De bit de fila
```

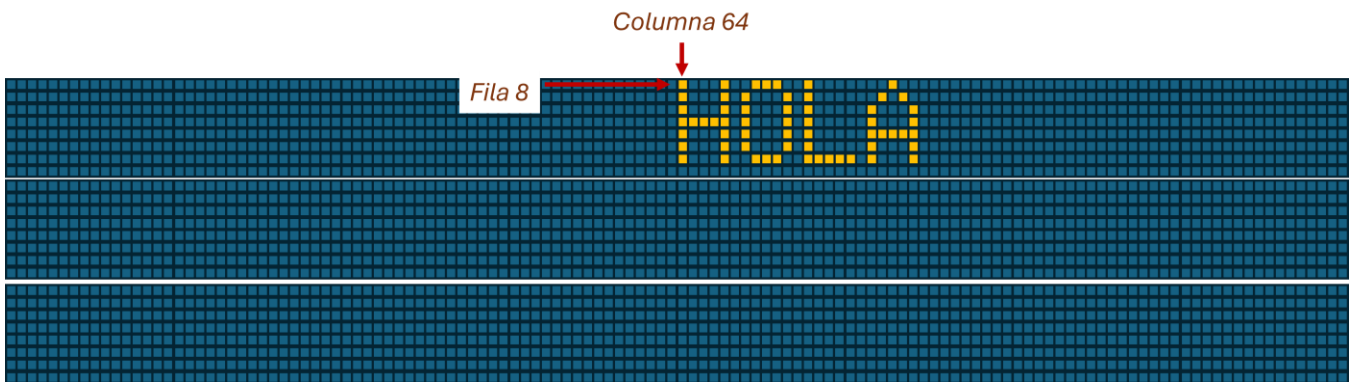


Figura 3.7-Impresión en la posición (64,0).

Otro ejemplo mostrado en la Figura 3.8.

```
display.setCursor(6,8); //No. De bit de Columna/No. De bit de fila
```

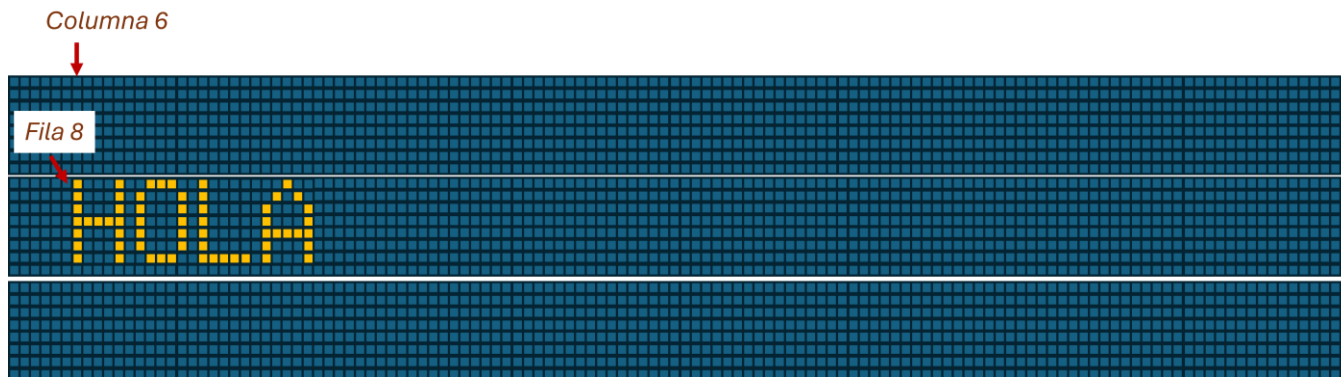


Figura 3.8-Impresión en la posición (6,8).

Algunas pantallas tienen las dos primeras páginas de color amarillo y los demás azules. Si se pone una dirección que abarque dos páginas de colores diferentes se vería como se muestra en la Figura 3.9.

```
display.setCursor(0,12); //No. De bit de Columna/No. De bit de fila
```

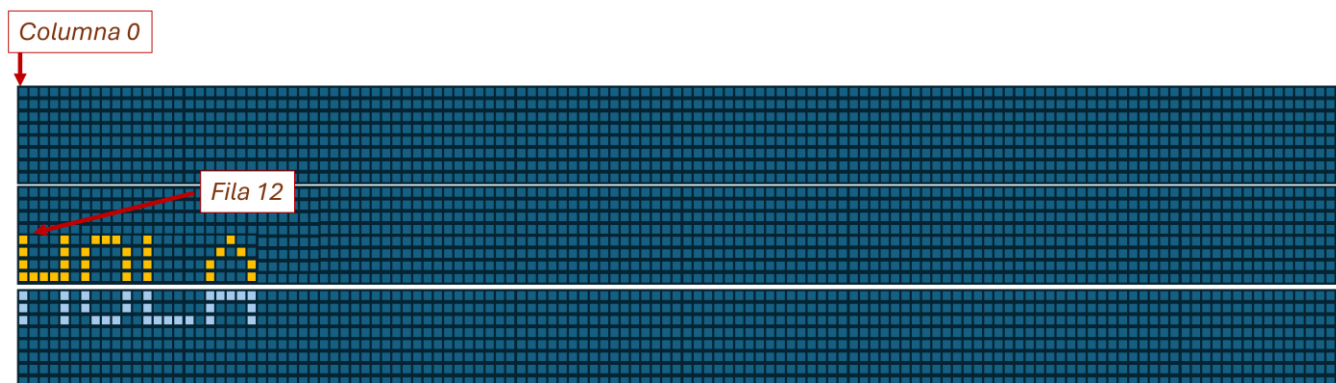


Figura 3.9-Impresión en la posición (0,12), en una pantalla con filas de color diferente.

Tamaño de fuente de la pantalla.

La librería de **Adafruit** permite cambiar el tamaño de la fuente con el siguiente comando:

```
display.setTextSize(size); //Define el tamaño de la fuente
```

El método `display.setTextSize(size);` acepta enteros positivos como argumento, por ejemplo:

- `display.setTextSize(1)`-Este es el tamaño de la fuente por defecto y los caracteres entran correctamente en una página. El tamaño de la fuente es de **5x7 píxeles**, como se muestra en la Figura 3.10.

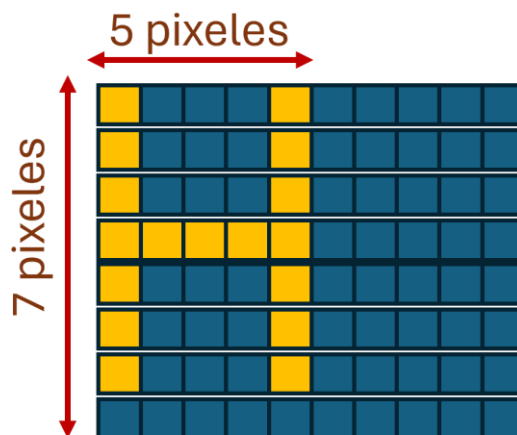


Figura 3.10-Fuente de tamaño 1.

- `display.setTextSize(2)`- Duplica el tamaño de la fuente. El tamaño de la fuente es de **10x14 píxeles**. En este caso el carácter necesitaría abarcar dos páginas. Además de remarcar el carácter con doble fila de píxeles, como se muestra en la Figura 3.11.

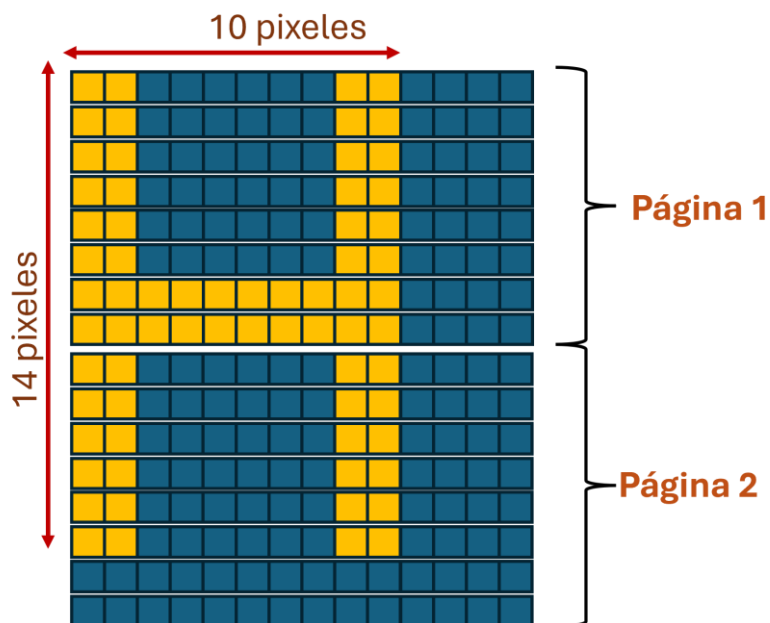


Figura 3.11-Fuente de tamaño 2.

- `display.setTextSize(3)`- El tamaño de la fuente es de **15x21 pixeles**. En este caso el carácter necesitaría abarcar tres páginas. Además de remarcar el carácter con triple fila de pixeles, como se muestra en la Figura 3.12.

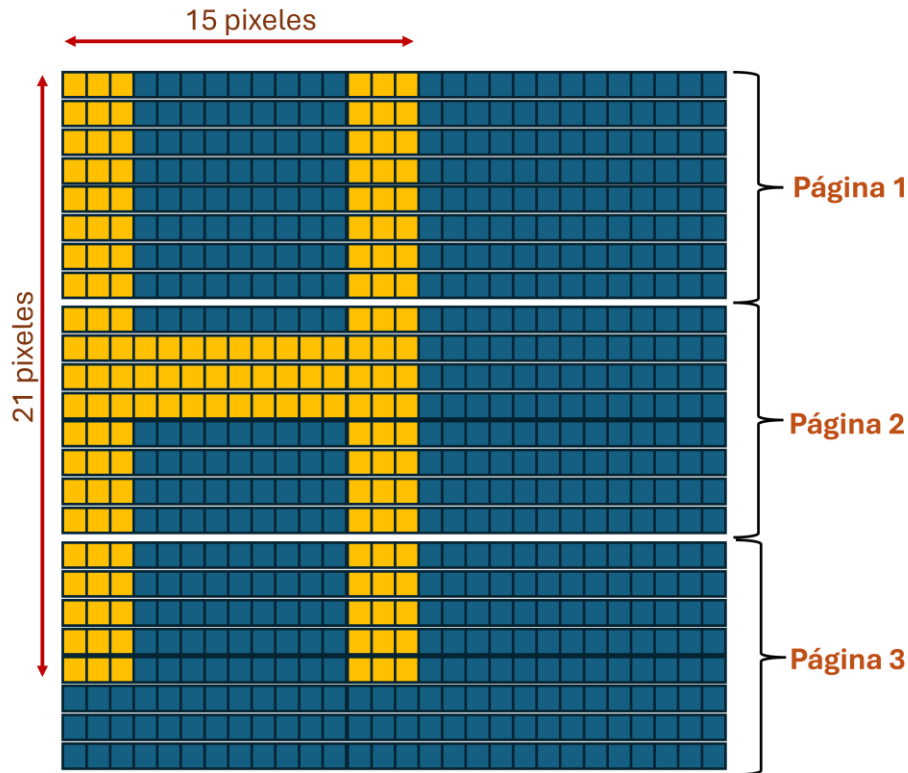


Figura 3.12-Fuente de tamaño 3.

Entonces tenemos que:

- `display.setTextSize(1)`- 5x7 px
- `display.setTextSize(2)`- 10x14 px
- `display.setTextSize(3)`- 15x21 px
- `display.setTextSize(n)`- (5xn)x(7xn) px

Puede observarse que el tamaño de la fuente puede aumentarse cuanto uno quiera, pero estará limitado por el tamaño físico de la pantalla.

Selección del color de los caracteres en pantalla

El comando `display.setTextColor();` sirve para definir el color de texto que se va a dibujar en pantalla. Pero no cambia el color del texto realmente, porque el color está definido por los LEDs que se encuentran en cada fila de la pantalla.

Pero se pueden usar los siguientes valores:

- `display.setTextColor(WHITE);` El texto se dibuja de color sobre fondo oscuro, Figura 3.13.

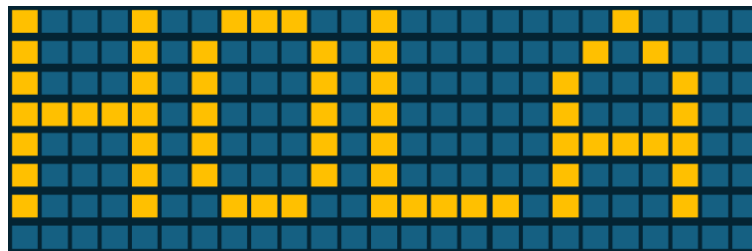


Figura 3.13-Fuente con configuración "White".

- `display.setTextColor(BLACK);` El texto se dibuja oscuro (útil para borrar texto), Figura 3.14.

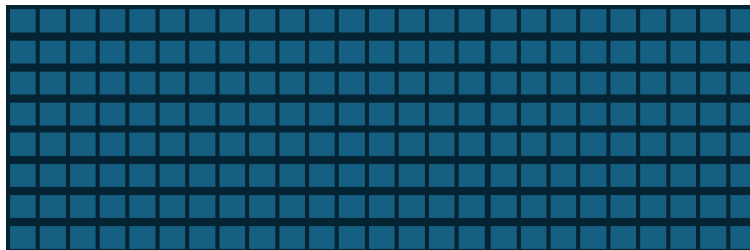


Figura 3.14-Fuente con configuración "Black".

- `display.setTextColor(BLACK, WHITE);` El texto se invierte. Oscuro sobre fondo de color, Figura 3.15.

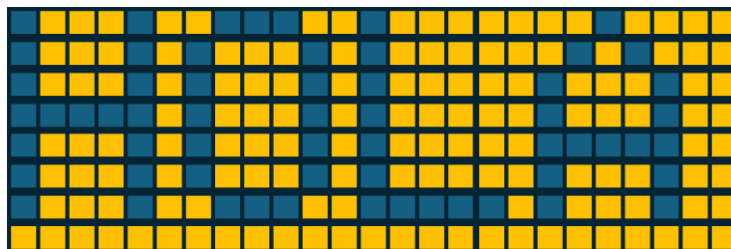


Figura 3.15-Fuente con configuración "Black-White".

Desplazar un texto

Para implementar un texto en movimiento existen varias formas. La más sencilla es usar los comandos:

- `display.startscrollright(start_page, stop_page);` Inicia un movimiento de izquierda a derecha.
- `display.startscrollleft(start_page, stop_page);` Inicia un movimiento de derecha a izquierda.

Donde:

- `start_page`: Página de memoria donde empieza el scroll (en este caso, la primera línea de la pantalla).
- `stop_page`: Página donde termina el scroll (en este caso, la última línea de la pantalla).

Estos comandos se representan en la Figura 3.16.

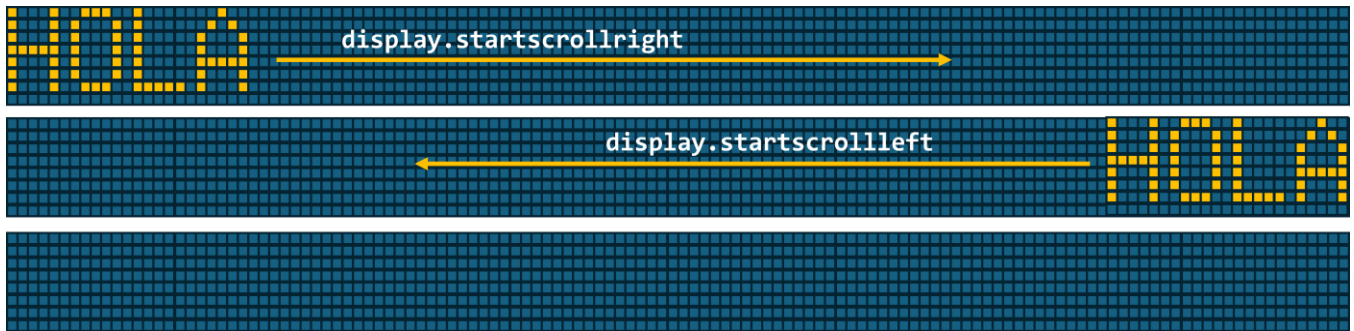


Figura 3.16-Movimiento de texto usando el comando `display.startscrollright/left`.

El movimiento se repite como si estuviera en la función `void loop()`. Aunque no pongas nada en el `loop()`, el scroll sigue ocurriendo en bucle, pero es el comportamiento normal del hardware del SSD1306. Porque el scroll no lo hace el microcontrolador (ESP32), lo hace el chip SSD1306 internamente, una vez que le mandas el comando `startscrollright()`. “El SSD1306 tiene un “motorcito” interno para desplazamiento. Le mandas la orden una sola vez, y él se encarga de mover los bits de la RAM de video, en un bucle infinito, hasta que le digas que pare”

El scroll se detiene con el comando: `display.stopscroll();` esto envía el comando de “detén el scroll”, y el chip SSD1306 para el movimiento.

Dibujar figuras en la pantalla.

En las pantallas OLED puedes dibujar cualquier imagen que se te ocurra, ya que tienes un lienzo de 128 pixeles por 64. Para dibujar tienes que crear un arreglo especificando los pixeles que se activarán, puedes apoyarte del enlace <https://javl.github.io/image2cpp/> para convertir cualquier imagen en un mapa de bits. A lo largo de la práctica se explica lo anterior con un ejemplo.

3.2 Desarrollo de la práctica Guiada I

3.2.1 Material por utilizar.

- 1 tarjeta ESP32
- 1 Protoboard
- Una pantalla OLED 128X64 pixeles
- Cables de conexión.

3.2.2 Ejercicio 1. Generando texto en pantalla.

a) Implementa el diagrama de la Figura 3.17.

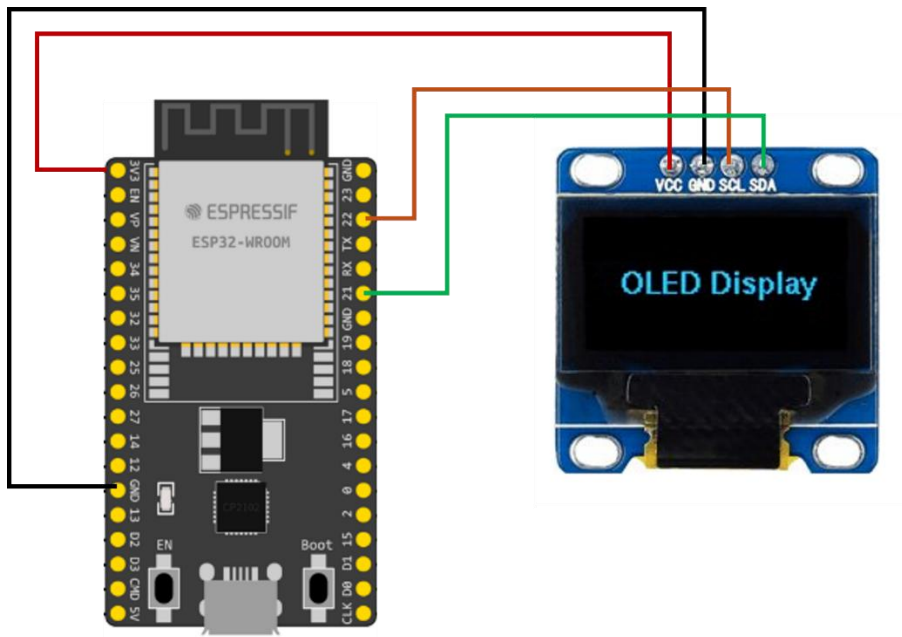


Figura 3.17-Conexión de la Pantalla OLED a la ESP32.

Captura el siguiente código para comprender las posiciones de pantalla, el tamaño de la fuente, los colores de texto y las animaciones.

```
#include <SPI.h>
#include <Wire.h>
//Librería que permite dibujar cosas básicas en pantalla, incluyendo texto.
#include <Adafruit_GFX.h>
//Librería que se encarga de trabajar directamente con el hardware de la pantalla
(driver)

#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128 // Ancho en píxeles de la pantalla.
#define SCREEN_HEIGHT 64 // Alto en píxeles de la pantalla.

// Declaration for SSD1306 display connected using I2C
#define OLED_RESET -1 // Pin de reset, este evita tener un botón de reset físico.
#define SCREEN_ADDRESS 0x3C //Dirección I2C de la pantalla.
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET); //Está
creando un objeto llamado display de la clase Adafruit_SSD1306, para controlar la
pantalla OLED usando el protocolo I2C.

// Variables para el círculo
#define ANCHO_PANTALLA 128
#define ALTO_PANTALLA 64

int x = 10;           // Posición horizontal inicial
int y = ALTO_PANTALLA/2; // Centro vertical de la pantalla
int radio = 10;       // Tamaño del círculo
int velocidadX = 2;   // Cuántos píxeles se mueve por cada paso

void setup()
{
  Serial.begin(9600);

  // Iniciando el objeto "display"
  if(!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;); // No procede, loop por siempre.
  }

  display.clearDisplay();           // Limpia pantalla
  display.setTextSize(3);           // Tamaño del texto
  display.setTextColor(WHITE);      //Color de texto
  display.setCursor(20, 16);        // Columna/fila-Posición donde se inicia la
  impresión de caracteres.
  display.println("YIZUS");          // Indica el texto que se va a imprimir "YIZUS"
  display.display();                // Imprime lo configurado.
  delay(2000);
```

```

display.clearDisplay();           // Limpia pantalla
display.setTextSize(2);          // Tamaño del texto
display.setTextColor(BLACK,WHITE); //Color de texto invertido
display.setCursor(36,24 );       // Columna/fila- Posición donde se inicia la
impresión de caracteres.
display.println("2025");          //Indica el texto que se va a imprimir "2025"
display.display();               // Imprime lo configurado.
delay(1000);

display.clearDisplay();          // Limpia pantalla
display.setCursor(0,16);        //Columna 0, fila 2.
display.setTextSize(1);         //Tamaño de letra 1
display.setTextColor(WHITE);    //Color de texto blanco
display.println("*****");
display.println("INTERNET");
display.println(" DE LAS ");
display.println(" COSAS ");
display.println("*****");
display.display();

//Scroll se activa en la página 3, 4 y 5. Deja estático los asteriscos
display.startscrollright(0x03, 0x05);
delay(12300);
display.stopscroll();
delay(1000);
}

void loop() {
display.clearDisplay();
display.drawCircle(x, y, radio, WHITE); // Dibujar el círculo en su nueva posición
display.display(); // Mostrar en pantalla
delay(30); // Esperar un poco para que se vea el movimiento
x += velocidadX; // Esperar un poco para que se vea el movimiento
// Si llega al borde derecho o izquierdo, cambiar la dirección (rebote)
if (x + radio >= ANCHO_PANTALLA || x - radio <= 0) {
    velocidadX = -velocidadX;
}
}

```

El siguiente código genera un texto en desplazamiento sin usar la función `display.startscrollright(INICIO,FIN)`; implementa el código en el circuito de la Figura 3.17 y analiza.

```

// Versión simplificada del código para mostrar "YIZUS" en una marquesina OLED
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

```

```

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET    -1
#define SCREEN_ADDRESS 0x3C

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
String palabra = "YIZUS"; // Texto a desplazar
int anchoTexto; // Ancho del texto en pixeles

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS);
  display.clearDisplay();
  display.setTextSize(1); // Tamaño mínimo del texto
  display.setTextColor(SSD1306_WHITE); // Texto blanco
  anchoTexto = palabra.length() * 6; // Cada letra ocupa 6 píxeles
}

void loop() {
  // Recorre el texto desde fuera del borde izquierdo hasta salir por la derecha
  for (int x = -anchoTexto; x <= SCREEN_WIDTH; x++) {
    display.clearDisplay();

    // Mostrar solo letras visibles (evita que salte a otra fila)
    int maxLetrasVisibles = (SCREEN_WIDTH - x) / 6;
    if (maxLetrasVisibles > palabra.length()) {
      maxLetrasVisibles = palabra.length();
    }
    if (x < SCREEN_WIDTH && maxLetrasVisibles > 0) {
      display.setCursor(x, 0); // Solo en la primera fila
      display.print(palabra.substring(0, maxLetrasVisibles));
    }
    display.display();
    delay(30); // Controla la velocidad del desplazamiento
  }
}

```

3.2.3 Ejercicio 2-Visualizando imágenes complejas en la pantalla OLED

a) Elige una imagen que sea de tu agrado, para este ejemplo se usará la Figura 3.18.



Figura 3.18-Imagen de Goku para mostrar en la pantalla OLED.

b) dirígete al siguiente enlace:

<https://javl.github.io/image2cpp/>

Cuando te encuentres dentro de la página selecciona “Elegir archivos” e inserta la imagen que seleccionaste “GokuNiño.jfif” en nuestro caso, como se muestra en la Figura 3.19.

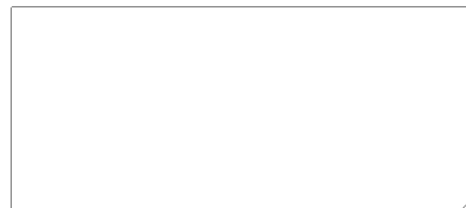
1. Seleccionar imagen

Todo el procesamiento se realiza localmente en su navegador; sus imágenes no se cargan ni se almacenan en ningún lugar en línea.

GokuNiño.jfif

o

1. Pegar matriz de bytes



128 incógnita 64 píxeles

Las imágenes leídas aparecen en el paso 3 a continuación.

Figura 3.19-Seleccionar imagen para mostrar en la pantalla OLED.

c) Configura la aplicación como se muestra en la Figura 3.20.

2. Image Settings

Canvas size(s): GokuNiño.jiff (file resolution: 195 x 259)
128 x 64 glyph remove ← Configuramos el tamaño de nuestra pantalla

Background color: ☒ White ☐ Black ☐ Transparent ← El color de lo caracteres

Invert image colors ☐

Dithering: Binary ▾

Brightness / alpha threshold: 128 ← Umbral de brillo de los LED, (255 es el brillo máximo)
0 - 255; if the brightness of a pixel is above the given level the pixel becomes white, otherwise they become black. When using alpha, opaque and transparent are used instead.

Scaling: scale to fit, keeping proportions ▾ ← Ajustar tamaño manteniendo proporciones.

Center image: ☒ horizontally ☐ vertically ← Centrar horizontal o verticalmente.
Centering the image only works when using a canvas larger than the original image.

Rotate image: 0 ▾ degrees ← Rotar Imagen

Flip image: ☐ horizontally ☐ vertically

Figura 3.20-Parámetros de configuración de la aplicación web.

d) La aplicación te ofrece una vista previa. En “Identificador/Prefijo” cambiamos el nombre para identificar mejor el código de la programación, como se muestra en la Figura 3.21.

4. Salida

Formato de salida del código: Código de Arduino ▾
Añade código Arduino adicional alrededor de la salida para facilitar su copia y pegado en [este ejemplo](#). Si se cargan varias imágenes, genera una matriz de bytes para cada una y añade un contador al identificador.

Identificador/Prefijo: Mapa_de_bits

Modo de dibujo: Horizontal - 1 bit por píxel ▾
Si la imagen se ve desordenada en la pantalla, como la imagen a continuación, intente usar un modo diferente.

Intercambiar bits en byte: ☐ intercambio
Útil cuando se trabaja con la biblioteca u8g2.

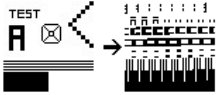


Figura 3.21-Vista previa de la imagen seleccionada.

- e) Seleccionamos “Generar código” y se genera un arreglo de número hexadecimales que prácticamente es la configuración de cada uno de los pixeles, en el lienzo de 129x64, Figura 3.22.

Generar código Copiar salida Descargar como archivo binario (.bin)

```
// 'GokuNino', 128x64px
const unsigned char Mapa_de_bitsGokuNino [] PROGMEM = {
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xef, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xe7, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xe1, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf1, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x8f, 0xf0, 0x7f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xc1, 0xf0, 0x3f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xe0, 0x70, 0x3f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xe0, 0x38, 0x3f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x18, 0x1f, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x00, 0x11, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
```

Figura 3.22-Código generado por la aplicación.

- f) Copia el código y sustitúyelo en la siguiente plantilla. El mostrado es el código para generar a “GokuNino”. Recuerda modificar el nombre de la imagen en el `void loop()`.

```
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET    -1
#define SCREEN_ADDRESS 0x3C
```

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

```
// 'GokuNino', 128x64px
const unsigned char Mapa_de_bitsGokuNino [] PROGMEM = {
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xef, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xe7, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xe1, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf1, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x8f, 0xf0, 0x7f, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xc1, 0xf0, 0x3f, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff,
```

0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xe0, 0x70, 0x3f, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xe0, 0x38, 0x3f, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x18, 0x1f, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x00, 0x11, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0x00, 0x00, 0x3f, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfc, 0x00, 0x00, 0x1f, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfc, 0x00, 0x00, 0x0f, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x1e, 0x00, 0x00, 0x07, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x06, 0x00, 0x00, 0x07, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x80, 0x02, 0x00, 0x03, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xc0, 0x02, 0x00, 0x03, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xc0, 0x00, 0x40, 0x00, 0x03, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xe0, 0x14, 0xc6, 0x00, 0x00, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x1d, 0xcc, 0x00, 0x03, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x1f, 0x9c, 0x40, 0x0f, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xd8, 0x1f, 0xbd, 0x40, 0x3f, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xe4, 0xff, 0xff, 0xc0, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xfd, 0xf0, 0xff, 0xff, 0x80, 0x07, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xd8, 0x78, 0xfe, 0xff, 0x00, 0x07, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xd0, 0xfe, 0xfe, 0xff, 0xe0, 0x3f, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0xfe, 0x2f, 0xdf, 0xe1, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xf1, 0xfe, 0x3f, 0xff, 0xe7, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xf9, 0xfe, 0x3f, 0xfb, 0x47, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfe, 0x1f, 0xfe, 0x1c, 0x7f, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfc, 0x17, 0xfc, 0x7e, 0x3f, 0xff, 0xff, 0xff,
0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfc, 0x0f, 0xf8, 0x7e, 0x37, 0xff, 0xff, 0xff,
0xff, 0xff,

0x00	0x01	0x02	0x03	0x04	0x05	0x06	0x07	0x08	0x09	0x0a	0x0b	0x0c	0x0d	0x0e	0x0f	0x10	0x11	0x12	0x13	0x14	0x15	0x16	0x17	0x18	0x19	0x1a	0x1b	0x1c	0x1d	0x1e	0x1f	0x20	0x21	0x22	0x23	0x24	0x25	0x26	0x27	0x28	0x29	0x2a	0x2b	0x2c	0x2d	0x2e	0x2f	0x30	0x31	0x32	0x33	0x34	0x35	0x36	0x37	0x38	0x39	0x3a	0x3b	0x3c	0x3d	0x3e	0x3f	0x40	0x41	0x42	0x43	0x44	0x45	0x46	0x47	0x48	0x49	0x4a	0x4b	0x4c	0x4d	0x4e	0x4f	0x50	0x51	0x52	0x53	0x54	0x55	0x56	0x57	0x58	0x59	0x5a	0x5b	0x5c	0x5d	0x5e	0x5f	0x60	0x61	0x62	0x63	0x64	0x65	0x66	0x67	0x68	0x69	0x6a	0x6b	0x6c	0x6d	0x6e	0x6f	0x70	0x71	0x72	0x73	0x74	0x75	0x76	0x77	0x78	0x79	0x7a	0x7b	0x7c	0x7d	0x7e	0x7f	0x80	0x81	0x82	0x83	0x84	0x85	0x86	0x87	0x88	0x89	0x8a	0x8b	0x8c	0x8d	0x8e	0x8f	0x90	0x91	0x92	0x93	0x94	0x95	0x96	0x97	0x98	0x99	0x9a	0x9b	0x9c	0x9d	0x9e	0x9f	0xa0	0xa1	0xa2	0xa3	0xa4	0xa5	0xa6	0xa7	0xa8	0xa9	0xaa	0xab	0xac	0xad	0xae	0xaf	0xb0	0xb1	0xb2	0xb3	0xb4	0xb5	0xb6	0xb7	0xb8	0xb9	0xba	0xbb	0xbc	0xbd	0xbe	0xbf	0xc0	0xc1	0xc2	0xc3	0xc4	0xc5	0xc6	0xc7	0xc8	0xc9	0xca	0xcb	0xcc	0xcd	0xce	0xcf	0xd0	0xd1	0xd2	0xd3	0xd4	0xd5	0xd6	0xd7	0xd8	0xd9	0xda	0xdb	0xdc	0xdd	0xde	0xdf	0xe0	0xe1	0xe2	0xe3	0xe4	0xe5	0xe6	0xe7	0xe8	0xe9	0xea	0xeb	0xec	0xed	0xee	0xef	0xf0	0xf1	0xf2	0xf3	0xf4	0xf5	0xf6	0xf7	0xf8	0xf9	0xfa	0xfb	0xfc	0xfd	0xfe	0xff
------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------

```

    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf1, 0xff, 0x1f, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfb, 0xff, 0x1f, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x3f, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x3f, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xbf, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff
};

```

```

// Array of all bitmaps for convenience. (Total bytes used to store images in PROGMEM =
1040)

```

```

const int Mapa_de_bitsallArray_LEN = 1;
const unsigned char* Mapa_de_bitsallArray[1] = {
    Mapa_de_bitsGokuNino
};

```

```

void setup()
{
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C); // here the 0x3c is the I2C address,
check your i2c address if u have multiple devices.
    display.clearDisplay();
    delay(2000);
}

```

```

void loop()
{
    display.drawBitmap(0, 0, Mapa_de_bitsGokuNino, 128, 64, WHITE); //Coordenadas donde
se dibuja la pantalla, el nombre del array que contiene la imagen en formato BITMAP,
ancho y alto de la pantalla, color de la imagen.
    display.display();
}

```

El resultado del código se observa en la Figura 3.23

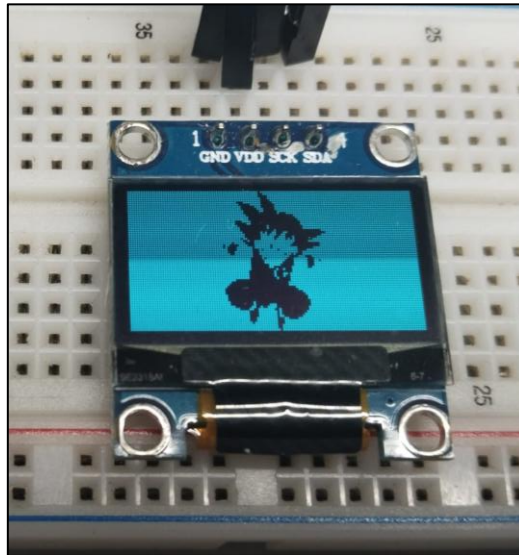


Figura 3.23-Imagen convertida en un mapa de bits mostrada en la pantalla OLED.

Condiciones de Entrega

El documento debe incluir los siguientes elementos:

- Portada
- Introducción (media cuartilla)
- Desarrollo del contenido
- Conclusiones

Convierte tu documento a formato **.pdf**, súbelo a la plataforma Microsoft Teams O Moodle. Utiliza el Acrobat 10 o posterior. No le pongas contraseñas ni protecciones a tu archivo.

Guarda todo tu documento con la siguiente estructura ***PACTICA_GUIADA3_Iniciales de los integrantes.***

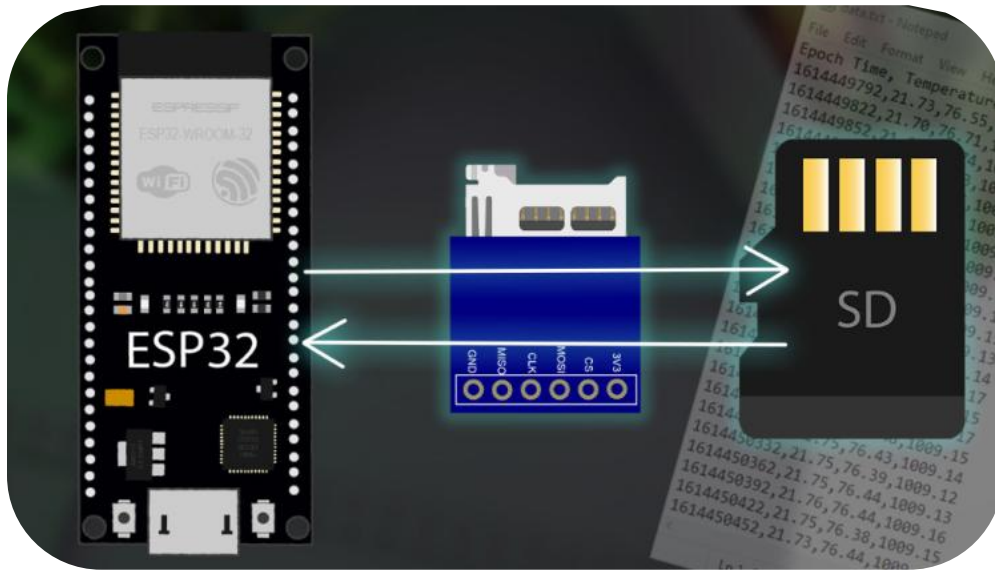
Escala de Evaluación

Carrera:	Ingeniería en Redes, Ingeniería en Sistemas de Energía e Ingeniería en Inteligencia Artificial.
Asignatura:	Introducción a Internet de las Cosas
Unidad(es) temática(s):	Unidad I
Actividad de Aprendizaje	Práctica Guiada 3. La pantalla OLED

Dimensiones o criterios a evaluar	Puntos por criterio	Puntos obtenidos	Observaciones
Realiza el ejercicio 1. Muestra todos los caracteres en pantalla. El funcionamiento es validado por el profesor.	1.5		
Agrega capturas de pantalla y fotografías como evidencia en el reporte. Del ejercicio 1.	1		
Realiza el ejercicio 2. Se muestra una palabra con desplazamiento.	1.5		
Agrega capturas de pantalla y fotografías como evidencia en el reporte. Del ejercicio 2.	1		
Realiza el ejercicio 3. Genera un mapa de bits en la pantalla OLED, a partir de una imagen.	2		
Agrega capturas de pantalla y fotografías como evidencia en el reporte. Del ejercicio 3.	1		
El documento cumple con la estructura solicitada, conteniendo todas las partes marcadas.	1		
Realiza una conclusión correcta del tema y de las competencias alcanzadas e indica lo aprendido y la manera de lograrlo.	1		
Total de Puntos	10		

*** Nota importante:** En caso de entregas fuera del tiempo marcado se aplicará una penalización de 1 punto menos a la calificación obtenida. Todos los integrantes deben de participar en la práctica o no se considerará en la calificación aquel que no se presente.

Práctica Guiada 4. Almacenamiento en medios extraíbles.



Objetivos:

- Implementar soluciones de IoT que incorporen sistemas de almacenamiento de datos en medios extraíbles.

Integrantes del equipo:

4.1 Marco teórico

4.1.1 Lector de Memoria microSD

Este módulo nos permite escribir información por medio de la tarjeta ESP32 a la tarjeta de memoria microSD, en forma de un documento de texto simple. Posteriormente esta información se puede analizar en un equipo de cómputo.

La tarjeta usa el estándar de comunicaciones SPI (del inglés Serial Peripheral Interface) es un estándar de comunicaciones, usado principalmente para la transferencia de información entre circuitos integrados en equipos electrónicos. El bus de interfaz de periféricos serie o bus SPI es un estándar para controlar casi cualquier dispositivo electrónico digital que acepte un flujo de bits serie regulado por un reloj (comunicación síncrona) (APRENDIENDO ARDUINO, 2025).

Incluye una línea de reloj, dato entrante, dato saliente y un pin de chip select, que conecta o desconecta la operación del dispositivo con el que uno desea comunicarse. De esta forma, este estándar permite multiplexar las líneas de reloj.

El SPI es un protocolo síncrono. La sincronización y la transmisión de datos se realiza por medio de 4 señales, ver Figura 4.1:

- **SCLK (Clock):** Es el pulso que marca la sincronización. Con cada pulso de este reloj, se lee o se envía un bit.
- **MOSI (Master Output Slave Input):** Salida de datos del Master y entrada de datos al Esclavo. También llamada SIMO.
- **MISO (Master Input Slave Output):** Salida de datos del Esclavo y entrada al Master. También conocida por SOMI.
- **SS/Select:** Para seleccionar un Esclavo, o para que el Master le diga al Esclavo que se active. También llamado chip Select (**CS**).

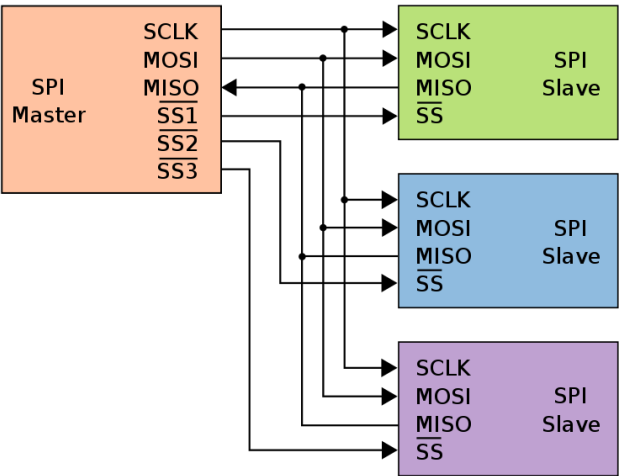


Figura 4.1-Protocolo SPI (APRENDIENDO ARDUINO, 2025).

En la Figura 4.2 se muestran los pines de conexión para usar el protocolo SPI:

Módulo de tarjeta microSD	ESP32
3V3	3,3 V
CS	GPIO5
MOSI	GPIO23
CLK	GPIO18
MISO	GPIO19
TIERRA	TIERRA

Figura 4.2-Pines con soporte para protocolo SPI

4.2 Desarrollo de la práctica

4.2.1 Material a utilizar.

- 1 Protoboard
- Pinzas de corte.
- Cables de conexión.
- 1 Módulo lector de memoria MicroSD
- 1 tarjeta ESP32
- 1 Memoria microSd
- Un adaptador para memoria microSD a SD.
- Un sensor de temperatura DS18B20.

4.2.2 Ejercicio 1. Creando un archivo de tipo texto para su almacenamiento en una memoria microSD

- a) Realiza la conexión entre los componentes como se muestra en la Figura 4.3.

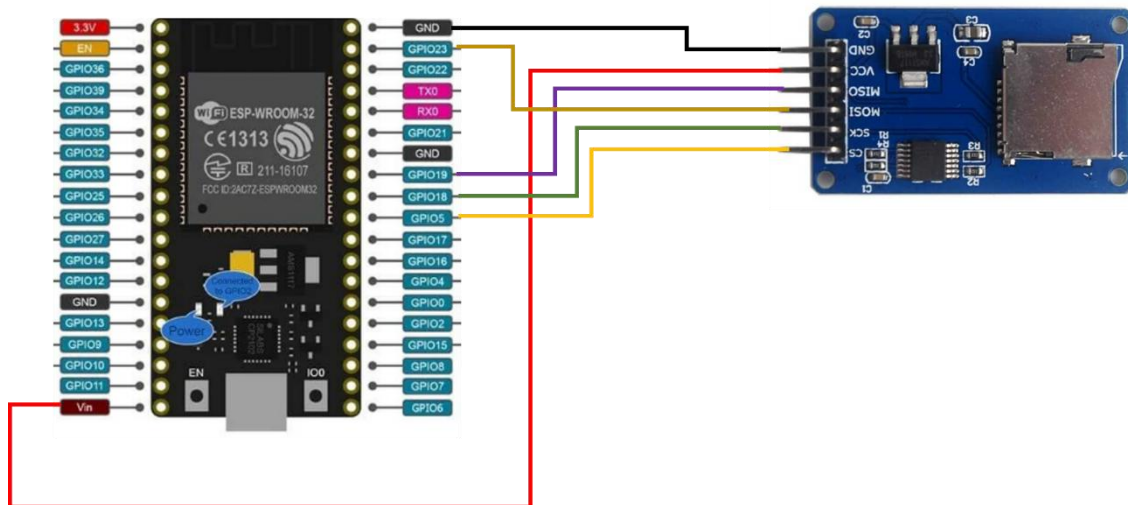


Figura 4.3-Conexión de la ESP32 al modulo lector de memorias SD.

b) Captura y analiza el siguiente código

```
#include "FS.h"
#include "SD.h"
#include <SPI.h>

#define SD_CS 5

void setup() {
    Serial.begin(115200);
    // Inicializando la tarjeta SD

    if(!SD.begin(SD_CS)) {
        Serial.println("No esta insertada la tarjeta");
        return;
    }
    uint8_t cardType = SD.cardType();

    if(cardType == CARD_NONE) {
        Serial.println("Sin tarjeta adjunta");
        return;
    }

    //Para poder ver el tipo de tarjeta
    Serial.print("SD Card Type: ");
    if(cardType == CARD_MMC){
        Serial.println("MMC");
    } else if(cardType == CARD_SD){
        Serial.println("SDSC");
    } else if(cardType == CARD_SDHC){
        Serial.println("SDHC");
    } else {
        Serial.println("UNKNOWN");
    }

    //Para obtener el tamaño de la tarjeta microSD
    uint64_t cardSize = SD.cardSize() / (1024 * 1024);
    Serial.printf("SD Card Size: %lluMB\n", cardSize);

    //Creando el archivo IOT.txt
    writeFile(SD, "/IOT.txt", "HOLA\n");
    //Agregamos datos al archivo IOT.txt
    appendFile(SD, "/IOT.txt", "MUNDO\n");
}

//Función para crear un archivo
void writeFile(fs::FS &fs, const char * path, const char * message){
    Serial.printf("Writing file: %s\n", path);

    File file = fs.open(path, FILE_WRITE);
    if(!file){
```

```

        Serial.println("Failed to open file for writing");
        return;
    }
    if(file.print(message)){
        Serial.println("File written");
    } else {
        Serial.println("Write failed");
    }
    file.close();
}

//Función para agregar contenido a un archivo o si es la primera vez lo crea
void appendFile(fs::FS &fs, const char * path, const char * message){
    Serial.printf("Appending to file: %s\n", path);

    File file = fs.open(path, FILE_APPEND);
    if(!file){
        Serial.println("Failed to open file for appending");
        return;
    }
    if(file.print(message)){
        Serial.println("Message appended");
    } else {
        Serial.println("Append failed");
    }
    file.close();
}

void loop(){
}

```

c) Agrega captura de pantalla del monitor serial donde se observe la información del archivo creado.

d) Agrega captura de pantalla del archivo almacenado en la memoria microSD.

4.2.3 Ejercicio 2. Medición de temperatura con sensor DS18B20

a) Realiza la conexión entre los componentes como se muestra en la Figura 4.4.

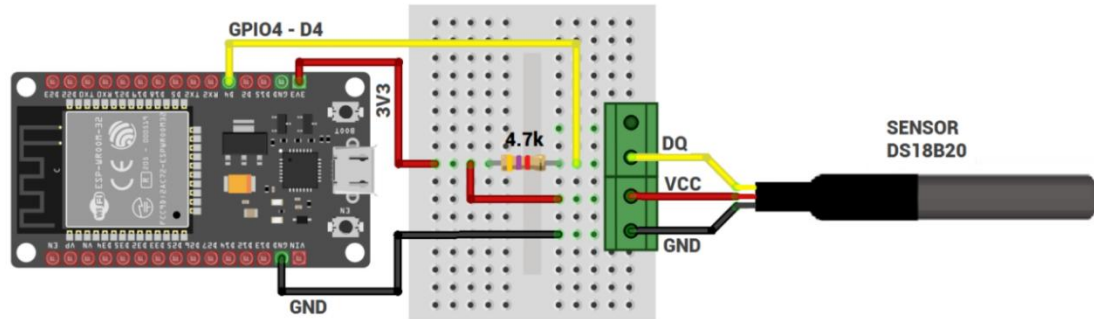


Figura 4.4-Conexión de la ESP32 al sensor de temperatura digital DS18B20

b) Captura y analiza el siguiente código.

```
#include <OneWire.h>
#include <DallasTemperature.h>

OneWire ourWire(4); //Se establece el pin 4 del ESP32 para la lectura del sensor
DallasTemperature DS18B20(&ourWire); //Se declara una variable u objeto para el sensor

void setup()
{
  Serial.begin(115200); //Se inicia el monitor serie a 115200 baudios
  DS18B20.begin(); //Se inicia el sensor
}

void loop()
{
  DS18B20.requestTemperatures(); //Se envía el comando para leer la temperatura
  float tem= DS18B20.getTempCByIndex(0); //Se obtiene la temperatura en °C

  Serial.print("Temperatura= ");
  Serial.print(tem);
  Serial.println(" °C");
  delay(1000);
}
```

c) Captura pantalla de la temperatura mostrada en el monitor serial.

4.4 Ejercicio 3. Almacenando datos de Temperatura usando sensor DS18B20.

- a) Realiza la conexión en conjunto del módulo de memoria SD y el sensor DS18B20. Como se muestra en la Figura 4.5.

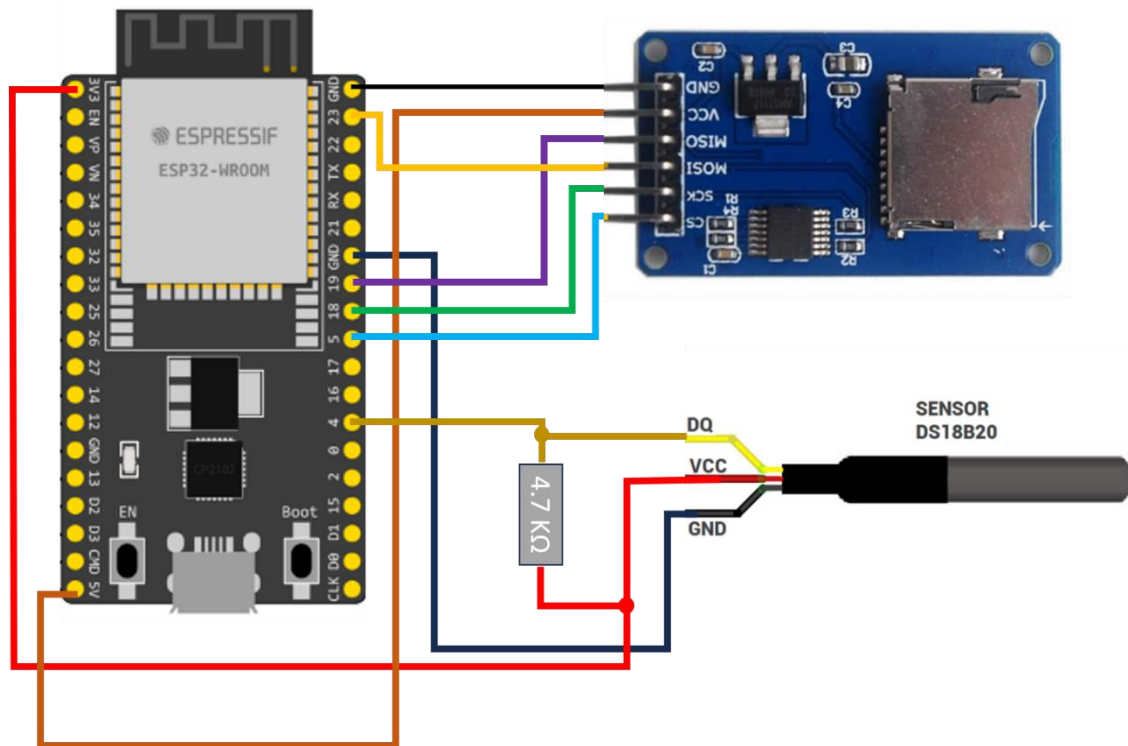


Figura 4.5-Circuito para el almacenamiento de datos de temperatura en la tarjeta SD.

- b) Captura y analiza el siguiente código.

```
#include <OneWire.h>
#include <DallasTemperature.h>

#include "FS.h"
#include "SD.h"
#include <SPI.h>

#define SD_CS 5
```

```
OneWire ourWire(4); //Se establece el pin 4 del ESP32 para la lectura del sensor
DallasTemperature DS18B20(&ourWire); //Se declara una variable u objeto para el sensor
```

```
int cont=0; //Variable entera que llevará la cantidad de mediciones realizadas.
String datos; //variable tipo string que servirá para almacenar los datos en la
memoria SD
float tempC=0; //variable global para la temperatura
```

```
void setup() {
```

```
//Inicializar la tarjeta SD
```

```
Serial.begin(115200);
```

```
DS18B20.begin(); //Se inicia el sensor
```

```
if(!SD.begin(SD_CS)) {
```

```
    Serial.println("No esta insertada la tarjeta");
```

```
    return;
```

```
}
```

```
uint8_t cardType = SD.cardType();
```

```
if(cardType == CARD_NONE) {
```

```
    Serial.println("Sin tarjeta adjunta");
```

```
    return;
```

```
}
```

```
//Esperamos un tiempo antes de iniciar las mediciones
```

```
delay(5000);
```

```
Serial.println("Iniciando Mediciones");
```

```
Serial.println("*****By YIZUZ*****");
```

```
delay (500);
```

```
}
```

```
void loop(){
```

```
//Código para la medición de temperatura.
```

```
DS18B20.requestTemperatures(); //Se envía el comando para leer la temperatura
```

```
tempC= DS18B20.getTempCByIndex(0); //Se obtiene la temperatura en °C
```

```
// Se imprime en el monitor serial
```

```
Serial.print("Temperatura: ");
```

```
Serial.print(tempC);
```

```
Serial.println(" °C");
```

```
delay(1000);
```

```
escribir_archivo(); //Llamamos a la función que escribirá los datos en el archivo
```

```
//Solamente se requiere 30 mediciones
```

```
if (cont>=30){
```

```
    Serial.println("Medición de temperatura finalizada");
```

```
    finalizar(); //Si se llega a 30 mediciones se llama a la función finalizar
```

```
}
```

```

}

void escribir_archivo(){
//Comando para convertir la variable tempC en un valor String
String datos = "temperatura," + String(tempC) + ",°C\n";
//Agregamos la medición al archivo tempC
appendFile(SD, "/tempC.txt",datos.c_str());
cont++; //Cada medición realizada aumenta el contador
return; //Regresamos a la siguiente línea de código del comando que llamó a la función
}

void finalizar(){
    Serial.println(".....Archivo Creado.....");
    while(cont>=30){} //Enciclamos en un ciclo while para cerrar el proceso.
}

//Función para agregar contenido a un archivo o si es la primera vez lo crea
void appendFile(fs::FS &fs, const char * path, const char * message){
//Serial.printf("Appending to file: %s\n", path);
File file = fs.open(path, FILE_APPEND);
if(!file){
    Serial.println("No se pudo abrir el archivo para agregarlo");
    return;
}
if(file.print(message)){
    //Serial.println("Mensaje adjunto");
} else {
    //Serial.println("Error al agregar");
}
file.close();
}

```

- c) Captura pantalla del monitor serial donde se observen las mediciones.
- d) Localiza el archivo almacenado en tu PC, como se muestra en la Figura 4.6.

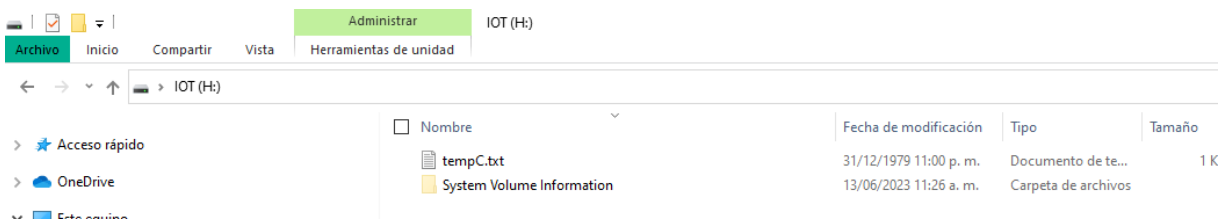


Figura 4.6-Archivo almacenado en tarjeta SD

- e) Abre el archivo tempC.txt en Excel y posteriormente te aparecerá la ventana de la Figura 4.7. Elige la configuración tal y como observas en la imagen.

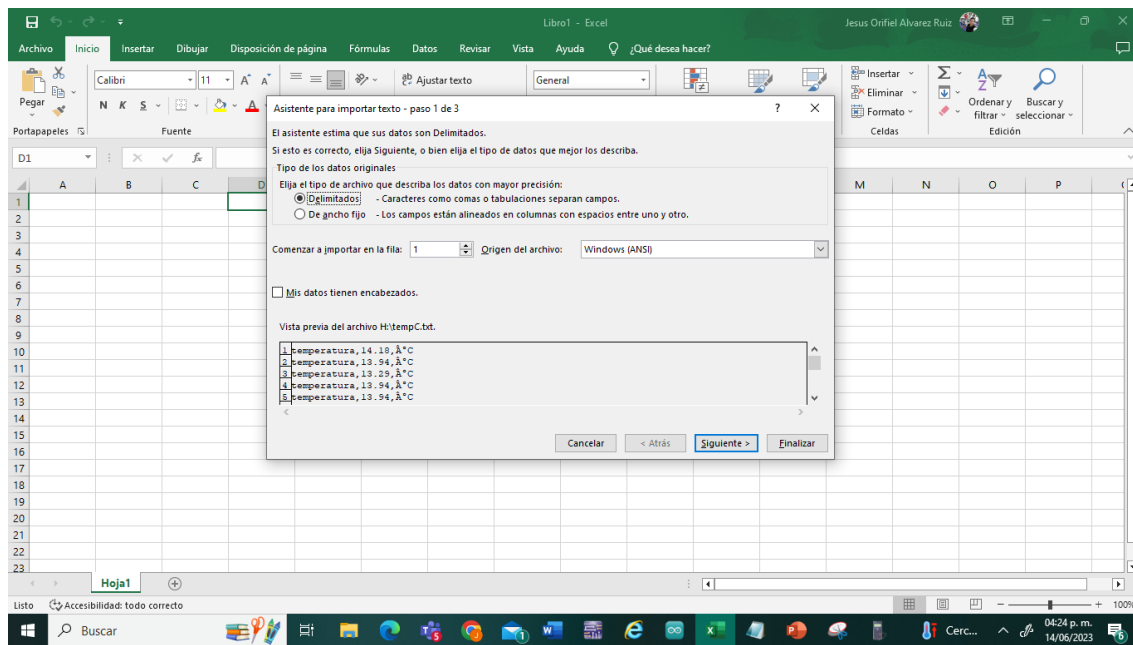


Figura 4.7-Configuración de Excel para abrir el archivo almacenado.

f) Elige separación con comas para separar las celdas, como se observa en la Figura 4.8.

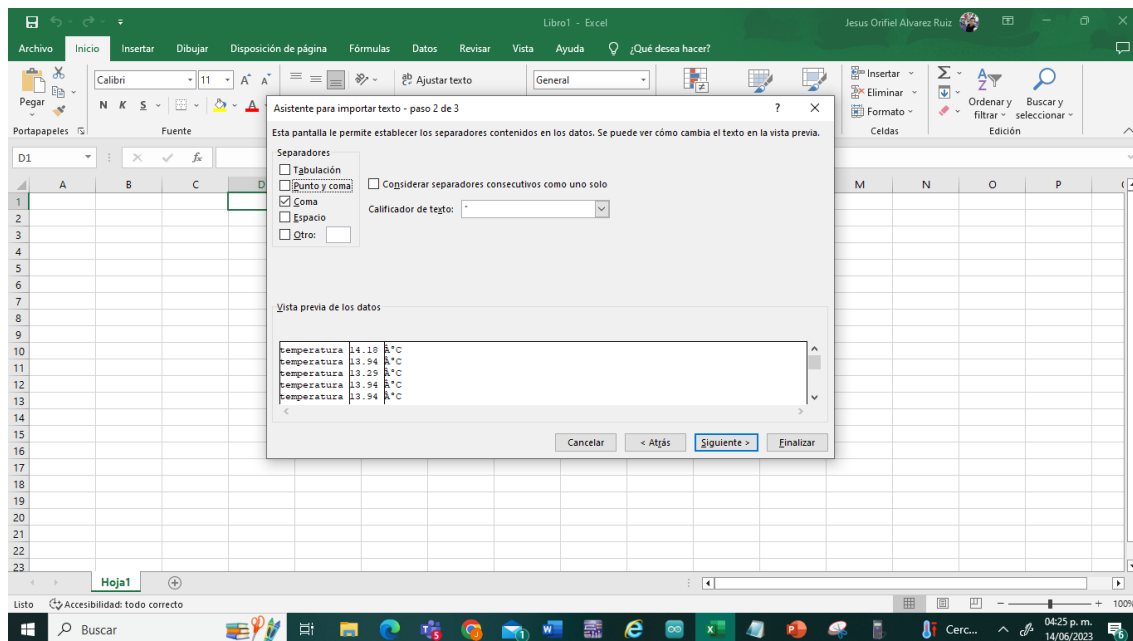


Figura 4.8-Selección de separación por comas para formar las celdas.

g) Clic en Finalizar, como se muestra en la Figura 4.9

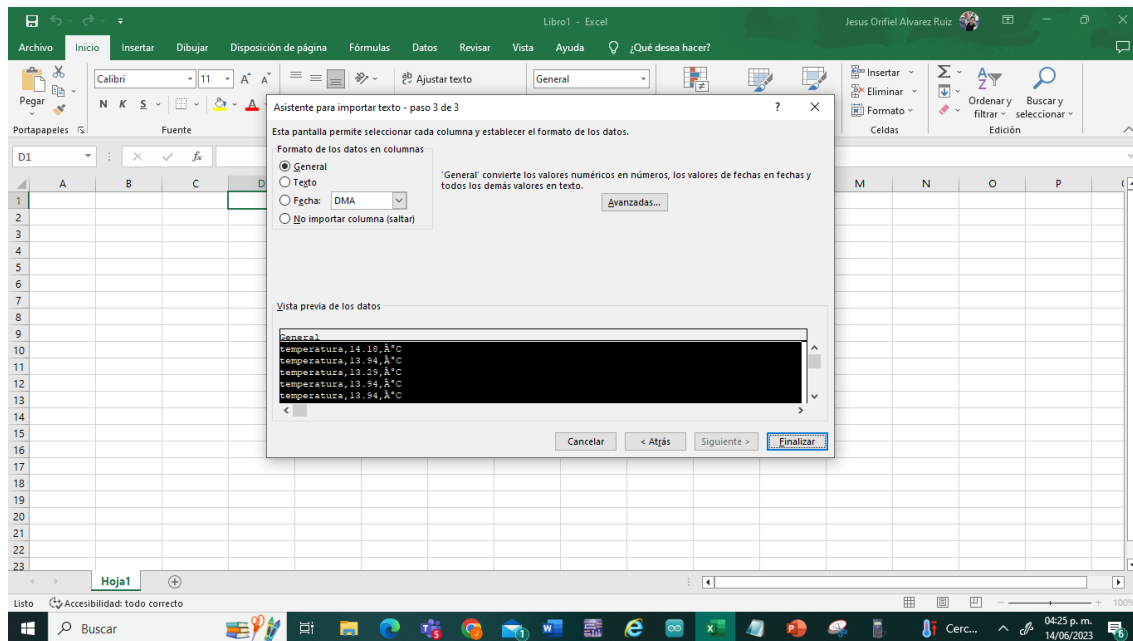


Figura 4.9-Finalización del asistente para mostrar información en Excel.

h) Si realizaste todo con éxito deberás poder ver una tabla con los valores de temperatura medidos como se observa en la Figura 4.10.

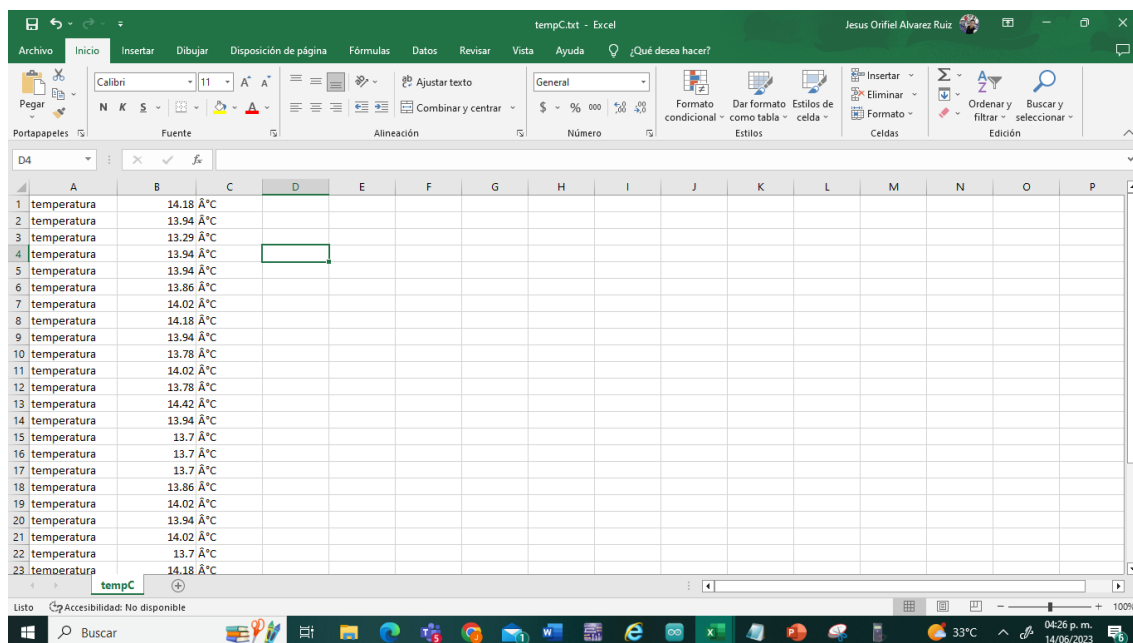


Figura 4.10-Información mostrada en Excel..

Condiciones de Entrega

El documento debe incluir los siguientes elementos:

- Portada
- Introducción (media cuartilla)
- Desarrollo del contenido
- Conclusiones

Convierte tu documento a formato **.pdf**, súbelo a la plataforma Microsoft Teams O Moodle. Utiliza el Acrobat 10 o posterior. No le pongas contraseñas ni protecciones a tu archivo.

Guarda todo tu documento con la siguiente estructura ***PACTICA_GUIADA4_Iniciales de los integrantes.***

Escala de Evaluación

Carrera:	Ingeniería en Redes, Ingeniería en Sistemas de Energía e Ingeniería en Inteligencia Artificial.
Asignatura:	Introducción a Internet de las Cosas
Unidad(es) temática(s):	Unidad I
Actividad de Aprendizaje	Práctica Guiada 4. Almacenamiento en medios extraíbles,

Dimensiones o criterios a evaluar	Puntos por criterio	Puntos obtenidos	Observaciones
Realiza el ejercicio 1. Genera el archivo .txt	2.5	2.5	
Realiza el ejercicio 2. Mide temperatura con el sensor LM35	2.5	2.5	
Realiza el ejercicio 3. Mide temperatura con el sensor LM35, la almacena en la tarjeta SD y lo importa a Excel.	3	3	
El documento cumple con la estructura solicitada, conteniendo todas las partes marcadas.	1	1	
Realiza una conclusión correcta del tema y de las competencias alcanzadas e indica lo aprendido y la manera de lograrlo.	1	1	
Total de Puntos	10	10	

* **Nota importante:** En caso de entregas fuera del tiempo marcado se aplicará una penalización de 1 punto menos a la calificación obtenida. Todos los integrantes deben de participar en la práctica o no se considerará en la calificación aquel que no se presente.

Práctica Guiada 5. Control de motores de CD y comunicación Serial.



Objetivos:

- Implementar dispositivos que hagan uso de motores de CD a los cuales se controlará su velocidad y dirección a través de comunicación serial.

Integrantes del equipo:

5.1 Marco teórico

El Motorreductor

Los reductores de velocidad son sistemas formados por engranajes que hacen que los motores eléctricos funcionen a distintas velocidades. Los reductores o motorreductores son necesarios para toda clase de máquinas y aparatos de uso industrial que precisan reducir de forma segura su velocidad. Los motorreductores ayudan a que el motor tenga más torque, es decir más fuerza. Un motorreductor industrial se muestra en la Figura 5.1.



Figura 5.1-Motorreductor Industrial (México, 2025).

El motorreductor que se usará en esta práctica se muestra en la Figura 5.2, este se alimenta de 5 a 9 V. Estos motores que funcionan con CD y tienen la característica de que al invertir la polaridad de sus terminales cambian de dirección. Como se muestra en la Figura 5.3.



Figura 5.2-Motorreductor para aplicaciones con Arduino y ESP32.

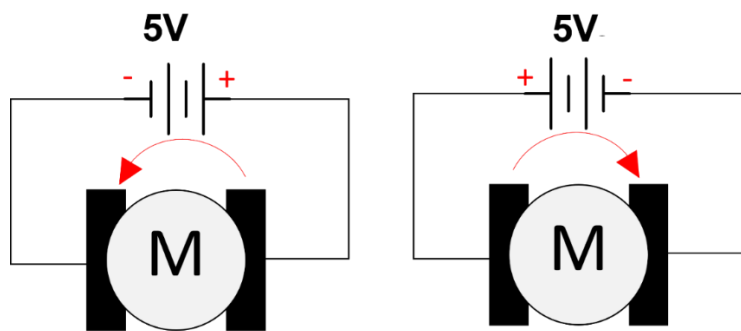


Figura 5.3-Cambio de dirección en un motor de CD.

Sin embargo, en la práctica no podrías cambiar manualmente el sentido de giro del motor, entonces es allí donde entra en acción el **punte H**, una configuración del puente H se muestra en la Figura 5.4.

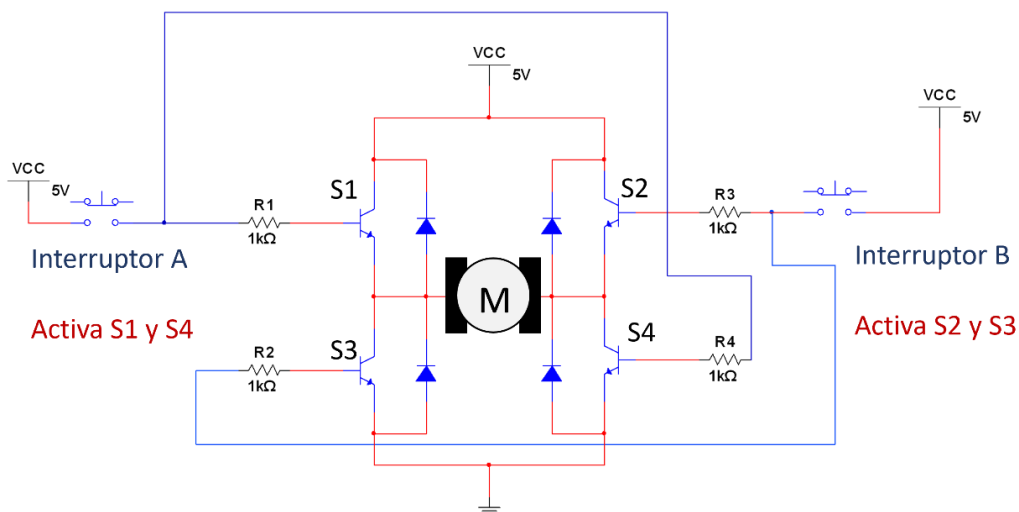


Figura 5.4- Configuración de un Puente H para control de motor de CD.

En un puente H, la regla es no activar dos interruptores de la misma rama, por ejemplo, si se activan S1 y S2 provocaría un corto circuito al unir el positivo con el negativo de la fuente. Por lo tanto, se deben activar dos interruptores al mismo tiempo, pero de manera cruzada, dependiendo del par de interruptores es el sentido en el que se mueve el motor, como se muestra en la Figura 5.5.

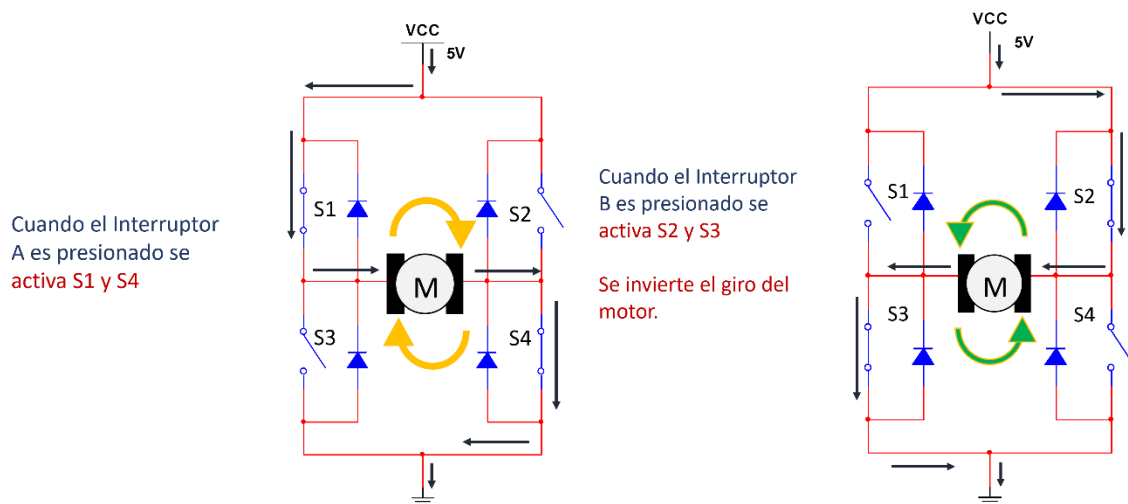


Figura 5.5- Inversión de giro de un motor de CD.

El componente **L298N** (Figura 5.6), tiene integrado dos módulos puente H. Es decir, puedes controlar dos motores al mismo tiempo.

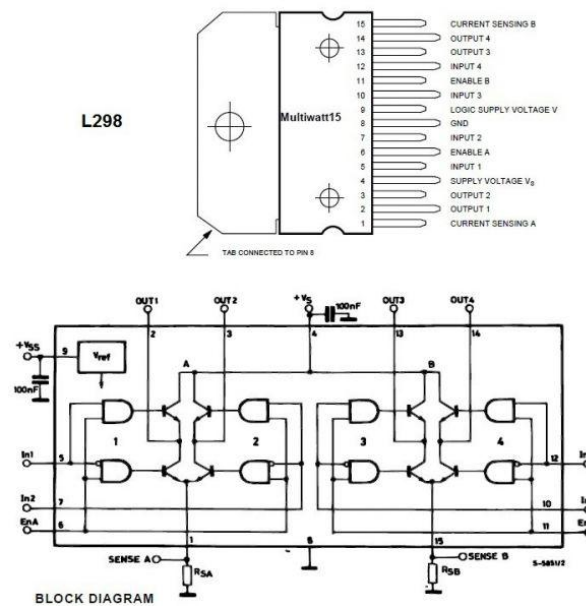


Figura 5.5- Circuito Integrado L298N (ALLDATASHEET, 2025).

El **L298N** viene integrado en el modulo que se muestra en la Figura 5.6. Este módulo se puede alimentar de 2 maneras gracias al regulador integrado LM7805.

Cuando el jumper de selección de 5V se encuentra activo, el módulo permite una alimentación de entre 6V a 12V DC. Como el regulador se encuentra activo, el pin marcado como +5V tendrá un voltaje de 5V DC. Este voltaje se puede usar para alimentar la parte de control del módulo ya sea un microcontrolador o un Arduino, pero recomendamos que el consumo no sea mayor a 500 mA.



Figura 5.6- Modulo puente H.

Cuando el jumper de selección de 5V se encuentra inactivo, el módulo permite una alimentación de entre 12V a 35V DC. Como el regulador no está funcionando, tendremos que conectar el pin de +5V a una tensión de 5V para alimentar la parte lógica del L298N (Electronilab, 2025). Usualmente esta tensión es la misma de la parte de control, ya sea un microcontrolador o Arduino, ver Figura 5.7.

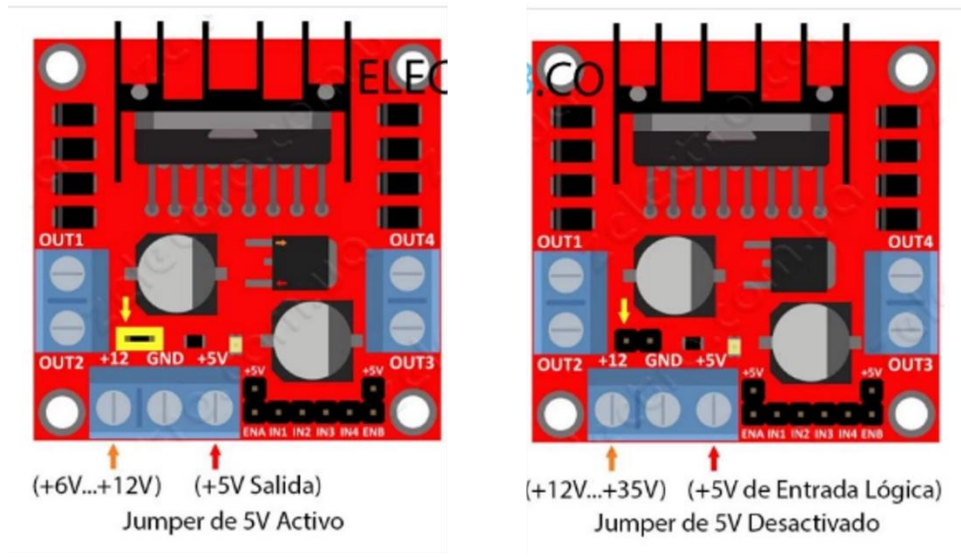


Figura 5.7- Terminales y voltajes de alimentación del módulo Puente H (Electronilab, 2025).

5.2 Desarrollo de la práctica

5.2.1 Material a utilizar.

- 1 Protoboard
- Una tarjeta ESP32
- 1 modulo puente H L298N

5.2.2 Ejercicio 1. Control de giro de un motor

a) Implementa el circuito de la Figura 5.8

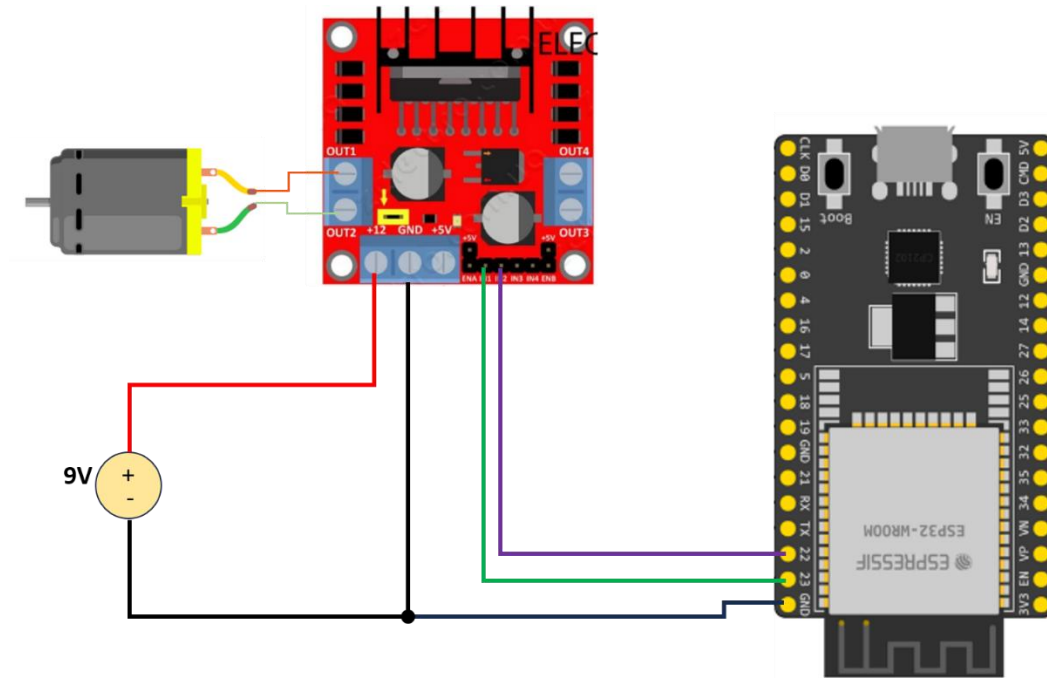


Figura 5.8- Conexión de la ESP32 al módulo puente H L298N.

b) Captura el siguiente código el cual realiza una rutina de cambio de dirección del motor.

```
#define IN1 23 //puerto para ir hacia adelante
#define IN2 22//puerto para variar la velocidad del motor PWM
void setup() {
  pinMode(IN1,OUTPUT);
  pinMode(IN2, OUTPUT);
}
void loop() {
  digitalWrite(IN1,HIGH);
  delay(1000);
  digitalWrite(IN1, LOW);
  delay(1000);
  digitalWrite(IN2,HIGH);
  delay(1000);
  digitalWrite(IN2,LOW);
  delay(1000);
}
```

5.2.3 Ejercicio 2. Control de velocidad del motor

a) Implementa el circuito de la Figura 5.9.

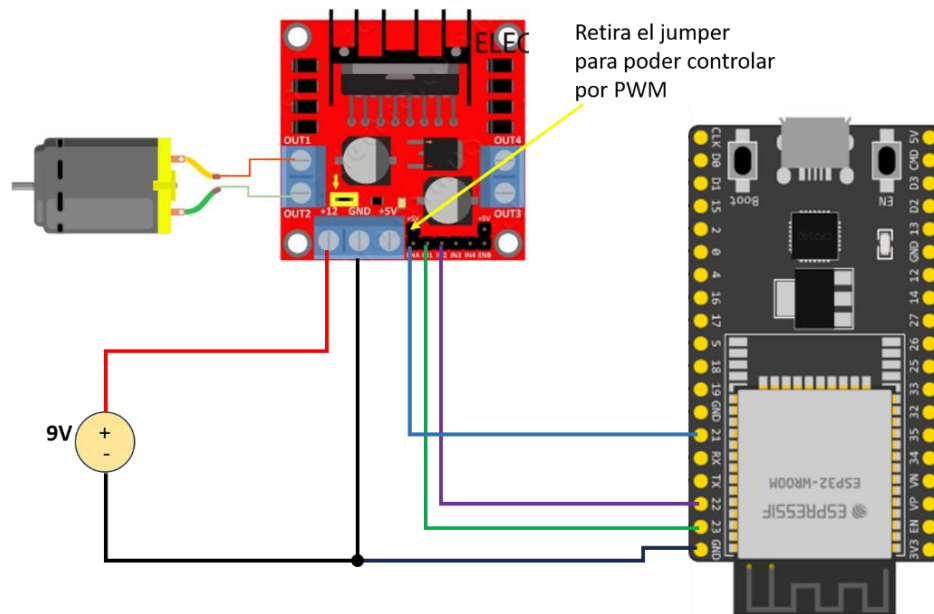


Figura 5.9- Conexión de la ESP32 al módulo puente H L298N para el control de velocidad de un motor.

c) Captura el siguiente código que realiza una rutina de control gradual de un motor de CD a través de comunicación serial.

```
#define IN1 23 //puerto para ir hacia adelante

char mensaje = 0;
int n = 0;
const int ledChannel = 0; // Canal PWM, puede ser de 0 a 15 const
int ENA1 = 21; // Pin al que está conectado el dispositivo const
int frequency = 5000; // Frecuencia en Hz
const int resolution = 8; // Resolución en bits (de 1 a 15)

void setup() {
  Serial.begin(9600);
  pinMode(IN1, OUTPUT);
  // ledcSetup(ledChannel, frequency, resolution);
  // ledcAttachPin(ENA1, ledChannel);
  ledcAttachChannel(ENA1, frequency, resolution, ledChannel);
}

void loop() {
  if (Serial.available() > 0) {
    mensaje = Serial.read();
    if (mensaje == '1') {
      n = 0;
      Serial.println("Motor apagado");
    }
  }
}
```

```

    }
    if (mensaje == '2') {
        n = 63;
        Serial.println("velocidad al 25%");
    }
    if (mensaje == '3') {
        n = 127;
        Serial.println("velocidad al 50%");
    }
    if (mensaje == '4') {
        n = 191;
        Serial.println("velocidad al 75%");
    }
    if (mensaje == '5') {
        n=255;
        Serial.println("velocidad al 100%");
    }
}
velocidad();
}

void velocidad() {
    digitalWrite(IN1, HIGH);
    ledcWrite(ENA1, n);
    delay(2000);
}

```

Condiciones de Entrega

El documento debe incluir los siguientes elementos:

- Portada
- Introducción (media cuartilla)
- Desarrollo del contenido
- Conclusiones

Convierte tu documento a formato **.pdf**, conservando la estructura del nombre señalado en los Criterios de Entrega de Actividades. Utiliza el Acrobat 10 o posterior. No le pongas contraseñas ni protecciones a tu archivo.

Guarda todo tu documento con la siguiente estructura
PACTICA_GUIADA5_niciales de los integrantes

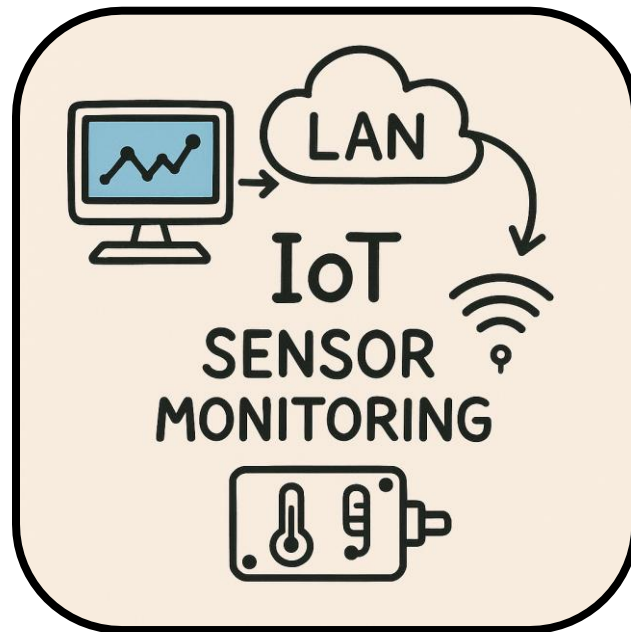
Escala de Evaluación

Carrera:	Tecnologías de la Información
Semestre	5
Asignatura:	Aplicaciones de IoT
Unidad(es) temática(s):	Unidad I
Actividad de Aprendizaje	Práctica Guiada 5. Control Serial

Dimensiones o criterios a evaluar	Puntos por criterio	Puntos obtenidos	Observaciones
Realiza el ejercicio 1. Es validado por el profesor en clase.	3	3	
Agrega capturas de pantalla y fotografías como evidencia en el reporte. Del ejercicio 1.	3	3	
Realiza el ejercicio 2. Es validado por el profesor en clase.	1	1	
Agrega capturas de pantalla y fotografías como evidencia en el reporte. Del ejercicio 2.	1	1	
El documento cumple con la estructura solicitada, conteniendo todas las partes marcadas.	1	1	
Realiza una conclusión correcta del tema y de las competencias alcanzadas interrelacionándolos e indica lo aprendido y la manera de lograrlo.	1	1	
Total de Puntos	10	10	

* **Nota importante:** En caso de entregas fuera del tiempo marcado se aplicará una penalización de 1 punto menos a la calificación obtenida.

Práctica Guiada 6. Sensor de temperatura y humedad en una red LAN.



Objetivos:

- Implementar un dispositivo que envíe datos de temperatura y humedad a una aplicación web.

Integrantes del equipo:

6.1 Marco teórico

6.1.2 Conceptos para entender la programación

HTTP: Abreviatura de la forma inglesa Hypertext Transfer Protocol, 'protocolo de transferencia de hipertextos'. En palabras simples es un protocolo que nos permite ver páginas web a través de Internet. El protocolo HTTP se puede ver en una URL al momento de buscar una página web, ver Figura 6.1.

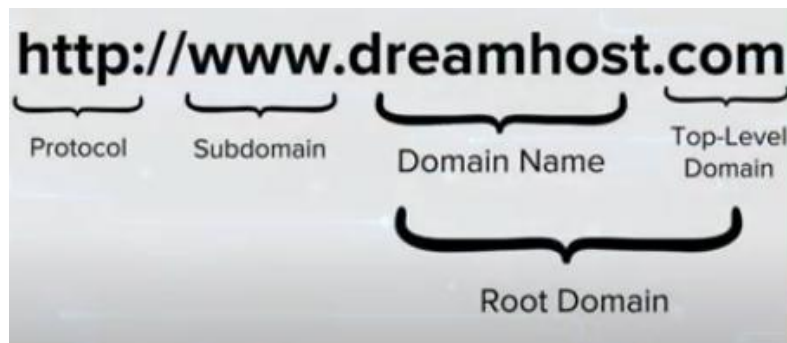


Figura 6.1- Estructura de una URL.

El protocolo HTTP está basado bajo un esquema cliente-servidor. Para esta el cliente es el navegador que se abre desde un dispositivo y el servidor es la tarjeta ESP32. En este caso el cliente se va a conectar a la dirección `http://192.168.1.71/` que asignamos en el código de arduino y la tarjeta ESP32 es el servidor que está en espera de las conexiones de los clientes, ver Figura 6.2.

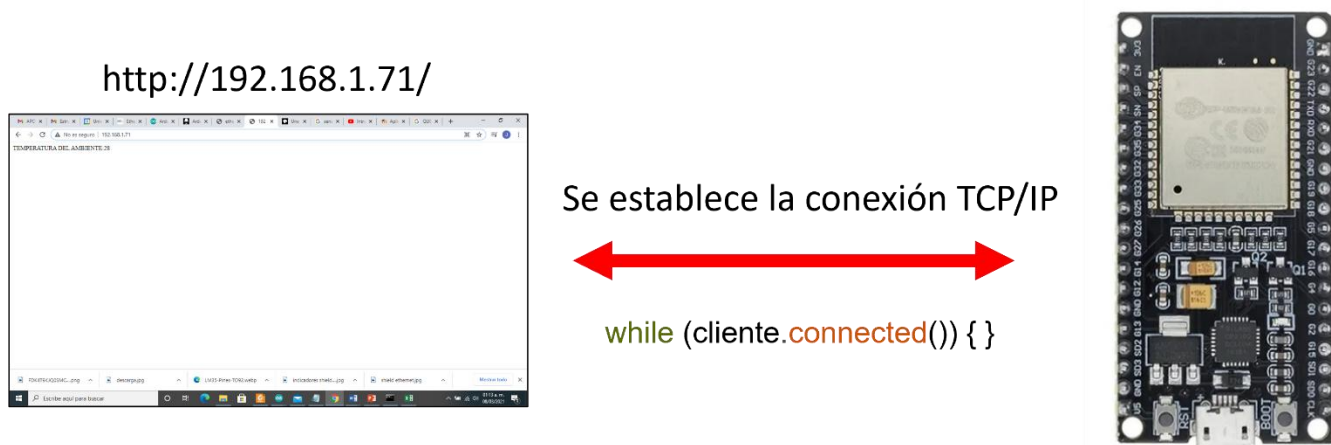


Figura 6.2- Esquema cliente-servidor

Una vez que se tiene una conexión ya abierta ya el cliente puede enviar un mensaje HTTP. En otras palabras, el cliente manda su petición, y espera por la respuesta. Los verbos definen la acción que se requiere realizar sobre el recurso, ver Figura 6.3. El verbo GET se usa para solicitar un recurso. Así que cada vez que en tu navegador ingresas una URL, por debajo el navegador está creando un mensaje que utiliza el verbo GET, aunque hay otros verbos. Esta parte no se ve en el código de Arduino, pero lo realiza la librería con los datos puestos (Cómo dirección Ip, dirección mac, entre otros).

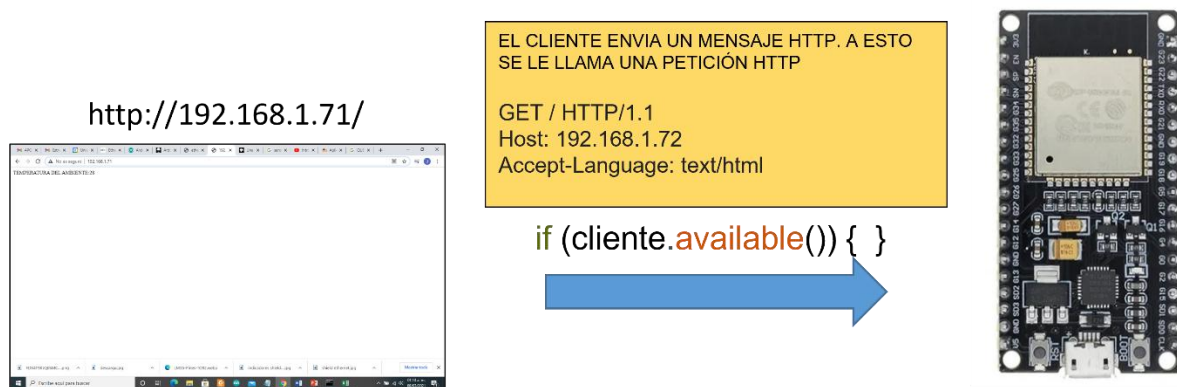


Figura 6.3- Envío de mensajes entre cliente-servidor

El servidor recibe el mensaje de petición y lo interpreta siguiendo las reglas del protocolo y se da cuenta que el cliente está haciendo una petición para acceder a la página 192.168.1.71 y crea un mensaje HTTP y responde, ver Figura 6.4. El código de respuesta de estado satisfactorio HTTP 200 OK indica que la solicitud ha tenido éxito

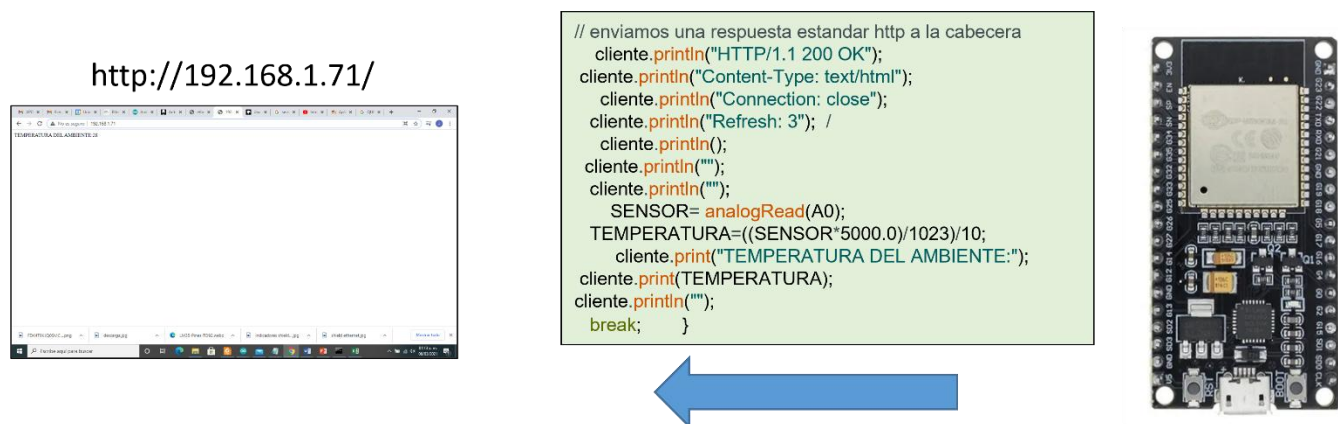


Figura 6.4- Ejemplo de un mensaje HTTP programado en Arduino IDE.

6.1.3 Sensor de temperatura y Humedad DHT11

El DHT11 presume de ser un sensor con una alta fiabilidad y estabilidad debido a su señal digital calibrada. Lo podemos comprar de dos maneras, de forma individual donde solo tenemos el sensor DHT11 o insertado en una PCB (Figura 6.5). La diferencia en precio no es excesiva y la versión con PCB aporta una resistencia pull-up de 5 k Ω y un LED que nos avisa de su funcionamiento. Otra diferencia entre estas dos versiones del DHT11 son los pines. Una de las ventajas que nos ofrece el DHT11, además de medir la temperatura y la humedad, es que es digital. A diferencia de sensores como el LM35, este sensor utiliza un pin digital para enviarnos la información y por lo tanto, estaremos más protegidos frente al ruido (Programarfacil, 2025).

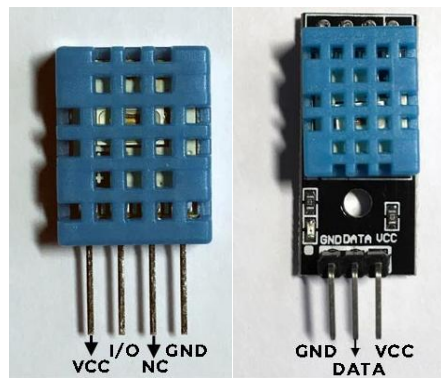


Figura 6.5- Sensor DHT11 en dos versiones comerciales.

Los pines de la versión con PCB del DHT11 son:

- GND: conexión con tierra
- DATA: transmisión de datos
- VCC: alimentación

Para este proyecto instala la librería que se muestra en la Figura 6.6.



Figura 6.6- Librería DHT11 de Adafruit.

6.2 Desarrollo de la práctica

6.2.1 Material a utilizar.

- 1 Protoboard
- 1 tarjeta EPS32
- 1 Sensor de temperatura y Humedad DHT11
- 1 LED
- 1 Resistencia de 330 Ohm
- Cables de conexión.

6.2.2 Ejercicio 1. Control de un LED vía web

a) Implementa el siguiente circuito de la figura 6.6.

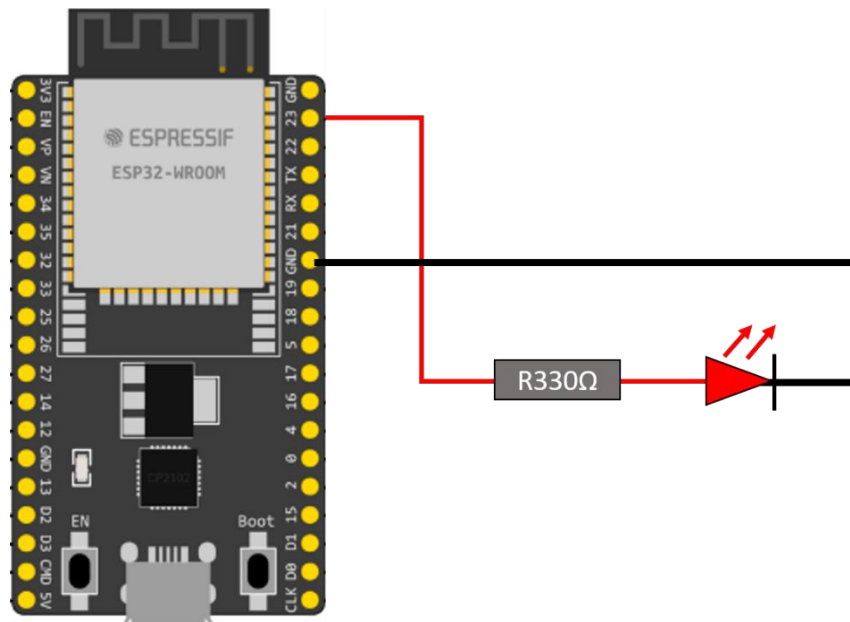


Figura 6.6- Control de un LED a través de una conexión WiFi.

b) Captura el siguiente código que permite controlar un LED a través de una red LAN.

```
#include <WiFi.h>
#include <WiFiClient.h>
#include <WebServer.h>
#include <DHT.h>

// Configuración del sensor DHT11
#define DHTPIN 2           // Pin donde está conectado el DHT11
#define DHTTYPE DHT11     // Tipo de sensor
DHT dht(DHTPIN, DHTTYPE);

// Configuración de la red WiFi
const char* ssid = "Linksys07982"; // Cambia por el nombre de tu red WiFi
const char* password = "3piyzkkr09"; // Cambia por tu contraseña

// Creación del servidor en el puerto 80
WebServer server(80);

void handleRoot() {
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    if (isnan(temperature) || isnan(humidity)) {
        server.send(500, "text/plain", "Error al leer el sensor DHT11");
        return;
    }

    String html = "<html><head><meta http-equiv='refresh' content='5'>";
    html += "<style>body{font-family:Arial; text-align:center;}</style></head><body>";
    html += "<h1>ESP32 - Monitor de Temperatura y Humedad</h1>";
    html += "<p>Temperatura: " + String(temperature) + " °C</p>";
    html += "<p>Humedad: " + String(humidity) + " %</p>";
    html += "</body></html>";

    server.send(200, "text/html", html);
}

void setup() {
    Serial.begin(115200);
    dht.begin();

    // Conexión a la red WiFi
    WiFi.begin(ssid, password);
    Serial.print("Conectando a WiFi...");
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.print(".");
    }
    Serial.println("\nConectado a la red WiFi");
    Serial.print("Dirección IP: ");
    Serial.println(WiFi.localIP());
}
```

```

// Configuración del servidor web
server.on("/", handleRoot);
server.begin();
Serial.println("Servidor web iniciado.");
}

void loop() {
  server.handleClient();
}

```

6.2.3 Ejercicio 2. Medición de temperatura y humedad por WiFi

a) Implementa el siguiente circuito de la Figura 6.7.

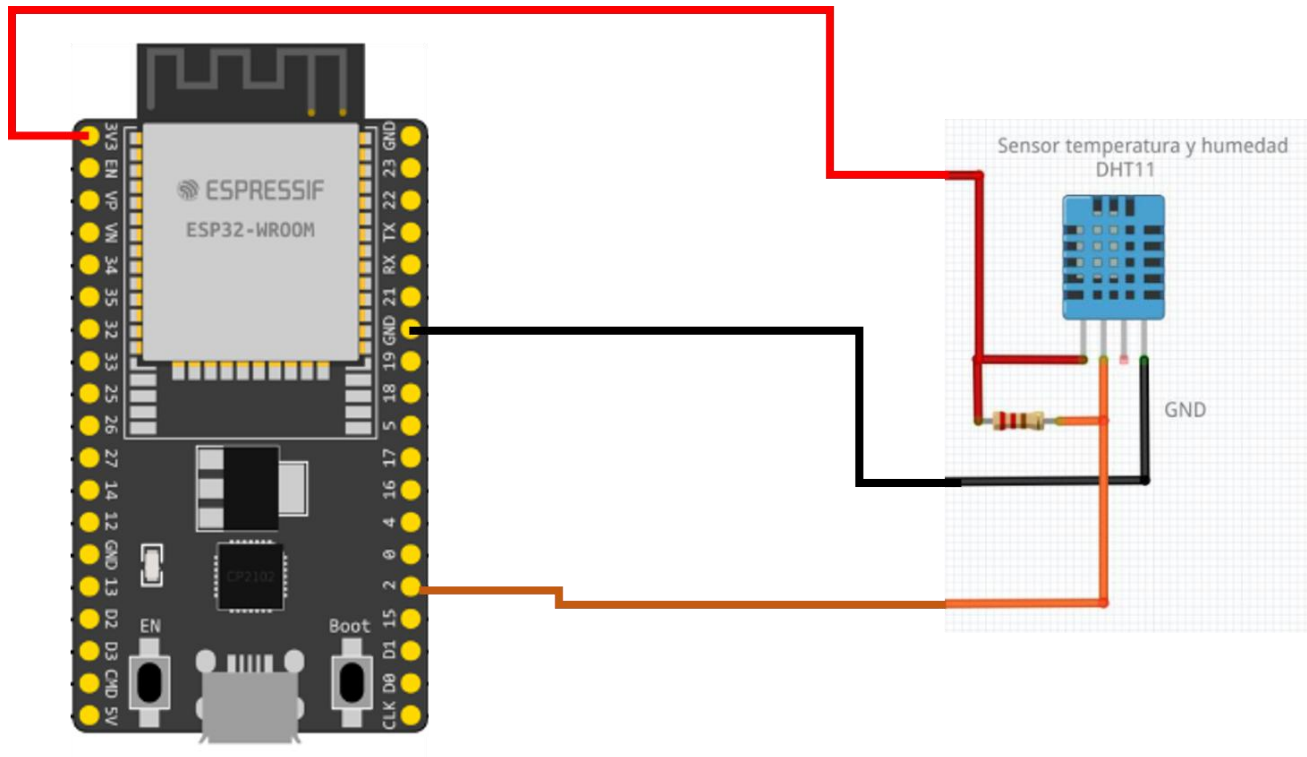


Figura 6.7- Circuito para el monitoreo de temperatura y humedad a través de una red WiFi.

- b) Captura el siguiente código el cual permite monitorear el valor de temperatura y humedad a través de una red WiFi.

```
#include <WiFi.h>
#include <WiFiClient.h>
#include <WiFiAP.h>

//Comandos y librerias para el funcionamiento del sensor.
#include <DHT.h>      // incluye libreria DHT de Adafruit
#include <DHT_U.h>    // incluye libreria Adafruit Unified Sensor

#define SENSOR 2      // Puerto D4 del nodemcu
int TEMPERATURA;      // variable para almacenar valor de temperatura
int HUMEDAD;          // variable para almacenar valor de humedad
int i=0;

/*Definimos nuestras credenciales de la red WiFi*/
const char* ssid = "Linksys07982";
const char* pass = "3piyzkkr09";

WiFiServer server(80);

DHT dht(SENSOR, DHT11); // objeto dht del tipo DHT en pin 4 y modelo DHT11

void setup() {

    dht.begin();        // inicializacion de sensor

    Serial.begin(115200);
    /*Iniciamos la conexión a la red WiFi definida*/
    WiFi.begin(ssid, pass);
    server.begin();
    delay(2000);
    /*En el terminal Serial, indicamos que se está realizando la conexión*/
    Serial.print("Se está conectando a la red WiFi denominada ");
    Serial.println(ssid);
    /*Mientras se realiza la conexión a la red, aparecerán puntos, hasta que se realice la conexión*/
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    /*Con la conexión ya realizada, se pasa a imprimir algunos datos importantes, como la dirección IP asignada a nuestro dispositivo*/
    Serial.println("");
    Serial.println("WiFi connected");
```

```

    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());
}

void loop() {
    // Esperando nuevas conexiones (clientes)
    WiFiClient client = server.available();

    if (client) {
        Serial.println("Nuevo cliente");
        // Boleano para localizar el fin de una solicitud http
        boolean blank_line = true;
        while (client.connected()) {
            if (client.available()) {
                char c = client.read();

                if (c == '\n' && blank_line) {

                    TEMPERATURA = dht.readTemperature(); // almacena en variable valor leído de
temperatura
                    HUMEDAD = dht.readHumidity(); // almacena en variable valor leído de
humedad

                    //Maquetación de la página con HTML
                    client.println("HTTP/1.1 200 OK");
                    client.println("Content-Type: text/html");
                    client.println("Connection: close");
                    client.println();
                    // Código para mostrar la temperatura y humedad en la página
                    client.println("<!DOCTYPE HTML>");
                    client.println("<html>");
                    client.println("<head></head><body><h1>YIZUS - ESP8266 - Humedad y
temperatura</h1><h3>Temperatura en Celsius: ");
                    client.println(TEMPERATURA);
                    client.println("*F</h3><h3>Humedad: ");
                    client.println(HUMEDAD);
                    client.println("%</h3><h3>");
                    client.println("</body>");
                    client.print("<meta http-equiv=\"refresh\" content=\"5\">"); //Actualización
de la página cada 5 segundos
                    client.println("</html>");
                    break;
                }
            }
            if (c == '\n') {
                // Cuando se empieza a leer una nueva línea
                blank_line = true;
            }
            else if (c != '\r') {
                // Cuando encuentra un caracter en la línea actual
                blank_line = false;
            }
        }
    }
}

```

```
}  
// Cerrando la conexión con el cliente  
delay(1);  
client.stop();  
Serial.println("Client disconnected.");  
}  
}
```

Condiciones de Entrega

El documento debe incluir los siguientes elementos:

- Portada
- Introducción (media cuartilla)
- Desarrollo del contenido
- Conclusiones

Convierte tu documento a formato **.pdf**, conservando la estructura del nombre señalado en los Criterios de Entrega de Actividades. Utiliza el Acrobat 10 o posterior. No le pongas contraseñas ni protecciones a tu archivo.

Guarda todo tu documento con la siguiente estructura ***PACTICA_GUIADA6_Iniciales de los integrantes***

Escala de Evaluación

Carrera:	Tecnologías de la Información
Asignatura:	Introducción a IoT
Unidad(es) temática(s):	Unidad I
Actividad de Aprendizaje	Práctica Guiada 6. Control WiFi

Dimensiones o criterios a evaluar	Puntos por criterio	Puntos obtenidos	Observaciones
Realiza el ejercicio 1. Es validado por el profesor en clase.	3		
Agrega capturas de pantalla y fotografías como evidencia en el reporte. Del ejercicio 1.	3		
Realiza el ejercicio 2. Es validado por el profesor en clase.	1		
Agrega capturas de pantalla y fotografías como evidencia en el reporte. Del ejercicio 2.	1		
El documento cumple con la estructura solicitada, conteniendo todas las partes marcadas.	1		
Realiza una conclusión correcta del tema y de las competencias alcanzadas interrelacionándolos e indica lo aprendido y la manera de lograrlo.	1		
Total de Puntos	10	8	

*** Nota importante:** En caso de entregas fuera del tiempo marcado se aplicará una penalización de 1 punto menos a la calificación obtenida.

Referencias

- ALLDATASHEET. (25 de Junio de 2025). Obtenido de <https://www.alldatasheet.es/datasheet-pdf/view/22440/STMICROELECTRONICS/L298N.html>
- APRENDIENDO ARDUINO. (25 de JUNIO de 2025). *Aprendiendo a manejar Arduino en profundidad*.
- ChatGPT-GPT-4o. (25 de Junio de 2025). Obtenido de Open AI.
- Electronilab. (25 de Junio de 2025). *Tutorial: Uso de Driver L298N para motores DC y paso a paso con Arduino*. Obtenido de <https://electronilab.co/tutoriales/tutorial-de-uso-driver-dual-l298n-para-motores-dc-y-paso-a-paso-con-arduino/?srsltid=AfmBOoqECDnMxIeFuIcsslM30XdgcnbxE5DZCC9PFqJt2XfLabL65NBM>
- FLOTANTE, P. (25 de JUNIO de 2025). *Pruebas al módulo sensor pasivo infrarrojo PIR HC-SR501*. Obtenido de <https://puntoflotante.net/MODULO-SENSOR-PASIVO-INFRARROJO-PIR-HC-SR501.htm>
- Llamas, L. (25 de Junio de 2025). *Ingeniería, informática y diseño*. Obtenido de <https://www.luisllamas.es/detector-de-metales-con-arduino-y-sensor-inductivo/>
- MECATRONICS, N. (26 de JUNIO de 2025). *Sensor Ultrasonido HC-SR04*. Obtenido de <https://naylampmechatronics.com/sensores-proximidad/10-sensor-ultrasonido-hc-sr04.html#:~:text=El%20sensor%20HC%20DSR04%20es,buena%20precisi%C3%B3n%20y%20excelente%20precio.>
- México, E. C. (25 de Junio de 2025). *¿Qué es un Motorreductor?* Obtenido de <https://www.energiacontrolada.com/faq/Que-es-un-motorreductor>
- Morgan, E. J. (16 de 11 de 2014). *ALLDATASHEET.COM*. Obtenido de <https://www.alldatasheet.com/datasheet-pdf/view/1132204/ETC2/HCSR04.html>
- NAYLAMP MECATRONICS. (25 de JUNIO de 2025). *Tutorial LCD, conectando tu arduino a un LCD1602 y LCD2004*. Obtenido de https://naylampmechatronics.com/blog/34_tutorial-lcd-conectando-tu-arduino-a-un-lcd1602-y-lcd2004.html